



BUILD KIT · VERSION 3.1 · KIMI CODE SWARM EDITION

OpenXR Operator

Strategy, architecture, adversarial QA, and a swarm-orchestrated production build plan for a local-first AI agent that observes and drives Godot XR applications.

Supersedes OpenXR Operator Build Kit v2.0 (8 Aug 2026); v3.0 (9 Aug 2026)

Date 9 August 2026 (v3.1: option B locality amendment)

Target engine Godot 4.7.1

Orchestration Kimi Code CLI swarm + Kimi K2.5/K2.6/K3 Agent Swarm

Licensing MIT in-engine / AGPL-3.0-or-later out-of-process

Vendor gate No Meta, OpenAI, or xAI. Google permitted.

Status Design audited, hardened, swarm-ready; implementation not started



Contents

- Part 0 — Executive summary: what v3.0 changes
- Part 1 — Strategy: the closed-loop operating model
- Part 2 — Target architecture (canonical, condensed)
- Part 3 — Research baseline and re-verification schedule
- Part 4 — v3.0 findings: seeded suspects, dispositioned
- Part 5 — The Kimi Code swarm operating model
- Part 6 — Build plan: waves, task cards, acceptance criteria
- Part 7 — Agent prompts (orchestrator and workers)
- Part 8 — Definition of done, decision register, open items
- Appendix A — Adversarial QA audit prompt v3.0 (full text)
- Appendix B — Production implementation prompt v2.0 (full text)
- References

Part 0 — Executive summary: what v3.0 changes

Build kit v2.0 fixed the fatal design flaws of v1.0 and shipped a sound architecture with an honest evidence ledger. Version 3.0 does not change that architecture. It does three things v2.0 did not do:

One. It audited v2.0 itself, adversarially. A v3.0 audit prompt (Appendix A) was built from a generic adversarial-QA template, hardened, and pointed at v2.0. That audit produced sixteen seeded suspects (S-01...S-16); fourteen are confirmed findings, two require build-time measurement. The highest-value: v2.0's CI workflow pins GitHub Actions by tag while its own gate mandates digest pinning (S-01); the only actuation tier shipping in v1 has no reference implementation (S-03); the audit log's "append-only" claim has no integrity mechanism (S-08); model weights are entirely absent from the supply-chain section (S-09); and the dependency register permits a gdUnit4 version incompatible with the target engine (S-12). Part 4 dispositions all sixteen.

Two. It re-verified every time-sensitive claim as of 9 August 2026. v2.0 was researched on 8 August 2026; its load-bearing claims were re-checked against primary sources (Part 3). The architecture's three load-bearers hold: Godot issue #100004 is still open, `EditorInterface` is still editor-only, and `WebSocketPeer` still exposes no inbound Origin header. Two claims needed updates: Godot 4.7.1 is now the current maintenance release (the kit pins 4.7.1 and states a 4.8 policy), and the Qwen3-VL caution note was re-dated.

Three. It makes the kit executable by a Kimi Code swarm. v2.0's kickoff prompt assumed a single agent working sequentially. v3.0 defines a swarm operating model (Part 5): one orchestrator session plus plane-owning worker sessions, coordinated entirely through ledger files in the repository, with contracts frozen before parallel work begins. This matches how Kimi Code actually works — each CLI session is an independent agent with its own context and transcript on disk; there is no native CLI swarm mode, so the swarm is a disciplined topology of sessions, not a feature flag^{[17][18]}. The same plan degrades gracefully to a single Kimi Code session using its built-in coder/explore/plan sub-agents, and scales up to Kimi's web Agent Swarm (100 sub-agents on K2.5, 300 on K2.6/K3)^{[19][21]}.

Verdict

OpenXR Operator is **READY WITH AMENDMENTS**. The gating amendments are the S-01/S-10 supply-chain fixes, the S-03 reference-implementation gap for A3 injection, the S-08 audit-log integrity mechanism, and the S-09 model-digest pinning — all specified in Part 6 as wave-0/wave-1 tasks. The concept is sound, the architecture survives audit, and the swarm plan is executable as written. Post-publication amendment (9 Aug 2026): the model plane gained the seam the engine plane already had — decisions D-10 (ModelBackend protocol + multiple adapters) and D-11 (vendor gate covers artifacts, not wire formats) are answered and recorded in Part 8; Ollama is demoted from the architecture to reference backend. Same-day amendment (option B): inference locality is decided as **local first, not local only** — D-12 records it, D-13 forbids automatic fallback. Hosted model providers are opt-in by manual configuration, one egress domain each, never an excluded vendor; the eight hosted adapters stay registered-but-unshipped until per-provider endpoint verification. The §3.6 egress trade-off is carried verbatim in Part 2.

0.1 How to use this kit

Reader	Read
Human deciding whether to build	Part 0, Part 4, Part 8 (decisions D-01...D-08 are yours)
Human running the build	Part 5 (swarm topology), Part 6 (waves and task cards), Part 7 (prompts to paste)
Orchestrator agent	Prompt P-0 (Part 7) plus Parts 1–4 as context
Worker agents	Their own prompt P-1...P-5 (Part 7) plus their contract files
Auditor agent (before or during build)	Appendix A, verbatim
Implementer agent (single-session fallback)	Appendix B, verbatim

0.2 Provenance conventions

This kit keeps v2.0's four epistemic states and adds one build-time state. Every non-obvious factual claim carries one:

Label	Meaning
CONFIRMED	Verified against a primary source (cited, dated) or verified by execution — for code claims, only a run counts.
INFERRED	Reasoned from primary evidence; reasoning shown.
BLOCKED	Retrieval was attempted and failed; what was tried and how it failed is recorded.
NOT ATTEMPTED	No retrieval was spent; stated plainly.
UNTESTED	Build-time claim not yet executed on the target host.

The distinction between BLOCKED and NOT ATTEMPTED is load-bearing and inherited unchanged from v2.0. Presenting one as the other is a finding against whoever writes it.

Part 1 — Strategy: the closed-loop operating model

1.1 The two loops

The strategy is two closed loops that share one ledger format. The **assurance loop** (Appendix A) finds everything that would fail before it is built. The **build loop** (Appendix B, swarm-orchestrated per Part 5) executes. They communicate through three machine-consumable artifact types:

Artifact	Produced by	Consumed by	Contents
F-### findings	Audit loop	Build loop	Severity, location, quoted evidence, fix, confidence. Kit v2.0 carries F-001...F-030; v3.0 audit starts at F-101.
A-## amendments	Audit loop	Build loop (first, in dependency order)	Exact document, exact change, findings resolved.
D-## decisions	Either loop	The human	Decisions only a human may make. D-01...D-08 inherited from v2.0; none assumed.

1.2 Standing rules (both loops, all agents)

Rule	Content
RPI	Research → Plan → Implement. No code changes until AUDIT.md and PLAN.md exist on disk.
Rule 9	No destructive or irreversible action without listing it first and getting explicit human approval.
Zero fabrication	Every claim backed by something run, read, or cited from a primary source, with the epistemic labels of §0.2.
Executable claims must be executed	A code claim verified by reading is INFERRED. Only a run makes it CONFIRMED.
Licence split invariant	addon/ and layer/ MIT; operator/ AGPL-3.0-or-later; repo default AGPL-3.0-or-later.
Vendor gate	No Meta, OpenAI, or xAI dependency — direct or transitive, including model lineage and training-data provenance. Google permitted. Enforced by tools/vendorscan.py in CI, not by intention.
Security gates invariant	Loopback-only binds; explicit environment-scoped OpenXR layer only; no browser-reachable path to the engine port; no mutating agent action without a human-approved diff. Each is a CI gate.

Supply chain	osv-scanner, never Trivy (GHSA-69fq-xp46-6x23). Actions pinned by commit digest. Downloaded binaries checksum-verified. Model weights pinned by digest and recorded in the SBOM.
Terse	Findings over narration. No hyperbole.

1.3 Why a swarm, and why this shape

The project has a natural parallel structure: four planes (GDScript addon, C++ layer, Python operator, web UI) with three narrow contracts between them (the wire protocol, the Driver protocol, the action schema). Contracts frozen first, planes built in parallel behind them — that is the textbook shape for parallel agents. Kimi K2.5/K2.6's Agent Swarm is trained for exactly this decomposition (PARL: a learned orchestrator fans out to frozen sub-agents, up to 100 on K2.5 and 300 on K2.6)^{[19][21]}, and Kimi Code CLI sessions are independent agents whose coordination medium is the filesystem^[17]. Part 5 turns those facts into a topology. The anti-goal is equally explicit: anything cross-cutting (CI gates, protocol changes, licence files) is owned by exactly one agent — the orchestrator — because two agents editing the same contract is how swarms fail.

Part 2 — Target architecture (canonical, condensed)

This part restates v2.0's architecture — which the v3.0 audit did not overturn — in the compressed form a swarm needs: planes, contracts, tiers, and gates. The full reasoning, including the observation problem (Godot issue #100004: viewport capture under `use_xr` returns black because the OpenXR compositor takes the render target)^[1] and the actuation problem, remains in v2.0 Parts 2–3 and is incorporated by reference.

2.1 Three planes, five contracts

Plane	Component	Licence	Owns
Instrumentation	addon/ — Godot addon (GDScript)	MIT	Authenticated loopback command server; scene-tree read; property get/set; T2 mirror capture; A3 tracker injection; editor bridge over the EngineDebugger capture channel
Observation/actuation T1/A2	layer/ — OpenXR API layer (C++17/CMake)	MIT	Explicit, environment-scoped layer; <code>xrReleaseSwapchainImage</code> / <code>xrEndFrame</code> hooks; Vulkan swapchain capture; layer-level pose injection. Optional, gated on D-04.
Control	operator/ — Python agent, flow runner, web UI	AGPL-3.0-or-later	Driver boundary; settle detector; selector grammar; YAML flows; action schema; ModelBackend adapters (plain httpx, no vendor SDK — D-10/D-11); approval gate; path containment; audit log; run artifacts; Origin-validated UI
Model	Model server behind the ModelBackend contract; local backends are the default, hosted providers opt-in by manual configuration, one egress domain each, never an excluded vendor (D-12/D-13)	—	Qwen2.5-VL-7B (default) or Gemma 3 12B; adapter-declared schema-constrained decoding; temperature 0

Every arrow between planes is loopback or in-process. No plane reaches the network except the configured model server (default 127.0.0.1:11434; the egress allowlist is deployment configuration, not a literal in the design — D-10) and the engine on 127.0.0.1:9099 — and the agent reaches nothing else, enforced *outside* the agent process (network namespace, firewall rule, or container policy). Inference is local first, not local only (D-12): local backends are the default and need no network permission; a hosted backend is off until a human edits configuration, opens egress to exactly one allowlisted provider domain, and is never an

excluded vendor (D-11). There is no automatic fallback between backends (D-13) — an unreachable backend fails with a named error. The trade-off, carried verbatim: *"Every hosted adapter you enable is a path by which a screenshot of your project and its scene tree leave the machine. That is the point of the feature and it is not a defect, but it means the egress allowlist is the control that decides how far a successful prompt injection travels."*

2.2 The five frozen contracts (the swarm's seams)

Contract	File(s)	Owned by	Change rule
Wire protocol v2 (auth-first, versioned, limits)	addon/openxr_operator/protocol.gd + docs	Orchestrator	Frozen after Wave 1; changes bump PROTOCOL_VERSION and touch both ends in one commit
Driver protocol	operator/src/openxr_operator/driver.py	Orchestrator	Frozen after Wave 1; the seam that makes capture/injection tiers interchangeable
Action schema	operator/src/openxr_operator/schema.py	Orchestrator	Single source of truth: passed to the model as format and used to validate the result
Flow YAML schema	flows/*.yaml + JSON Schema	Orchestrator	Validated before any step executes; capabilities default to empty (deny-all)
ModelBackend protocol	operator/src/openxr_operator/model.py + backends/	Orchestrator	The model-side seam mirroring Driver (D-10). Adapters declare their constrained-decoding mode; below "native" the run report must record the degraded posture. Resolution is fail-closed: the permanent excluded-vendor denylist is applied centrally, after allowlist resolution, and is not configuration (D-11/D-12)

2.3 Tier model (fidelity is data, not a claim)

Tier	Capture mechanism	Fidelity	Ships
T1	OpenXR API layer at xrReleaseSwapchainImage	Exact composited frame, per eye	v2 (D-04 default: defer)
T2	Non-XR mirror SubViewport, captured after RenderingServer.frame_post_draw	Approximates the scene; no distortion, no compositor post-processing	v1
T3	Desktop harness, use_xr = false	Full scene fidelity, zero XR fidelity	v1

Tier	Injection mechanism	Fidelity	Ships
A1	XR_EXT_conformance_automation via OpenXRAPIExtension	Full action pipeline; Monado supports it, most consumer runtimes do not ^{[11][12]}	where available
A2	API layer intercepting pose/action-state calls	Full pipeline below the application	with the layer
A3	In-engine XRControllerTracker fabrication, names constrained to left_hand/right_hand, with post-injection binding assertion	Godot node glue only	v1 (floor)

Every capture and injection carries its tier through the run artifact into the report. A test that passed on T2 has not proven what T1 proves, and the report must say which it was. Silent fidelity downgrade is a Blocker.

2.4 Security model (condensed)

Trust boundaries, decreasing: human operator; local agent process; Godot editor/game; local model server; model output; any open web page; the network. The last three are untrusted. Model output is data: schema-validated, allowlist-checked, and — for anything mutating — presented as a diff for human approval (never auto-applied in v1, D-06). The browser never touches the engine: the engine port requires token-first authentication (32 bytes from `Crypto.generate_random_bytes()`, constant-time compare, one attempt, 2-second deadline), and all browser-facing surface lives in the Python plane where Origin is readable. Chrome's Local Network Access enforcement (147+, April 2026) raises the bar in Chromium only and is not a control this design depends on^[4]. One agent step: settle → capture(tier) → scene tree → model.act(schema) → validate → allowlist → approval(diff) → execute → audit-log. Steps 9–11 of v2.0 §2.7 are the entire difference from v1.0's loop.

Part 3 — Research baseline and re-verification schedule

Verified 9 August 2026 against primary sources. This is the starting state; the orchestrator re-verifies every row on build day (the audit's Annex B schedule, Appendix A).

Table 3-1 Research baseline (R-1...R-13)

#	Claim	Status 9 Aug 2026	Source
R-1	Godot issue #100004 (black viewport under use_xr) open, unfixed	CONFIRMED open (filed 3 Dec 2024; Vulkan + OpenGL)	[1]
R-2	Godot 4.7.x is the right target	Updated: pin 4.7.1 (14 Jul 2026); state a 4.8 policy	[2][3]
R-3	Chrome 147 LNA WebSocket enforcement; Firefox/Safari do not enforce	CONFIRMED; LNA unusable as a dependable control	[4]
R-4	Ollama format accepts JSON Schema, works with vision models	CONFIRMED (cloud tier unsupported — irrelevant to local-only)	[5]
R-5	Qwen3-VL vision-detection issues (Mar 2026)	Partially stale: fixes recorded downstream; "verify your exact tag" stands	[6][7]
R-6	XR_EXT_conformance_automation: Monado implements it	CONFIRMED and strengthened (Monado 25.1.0; Khronos-funded upgrade project)	[11][12]
R-7	gdUnit4 "v6.x" in the dependency register	Imprecise — finding S-12: Godot 4.7/4.7.1 requires $\geq$ v6.2.0	[10]
R-8	WCAG 2.2 AA is the right accessibility baseline	CONFIRMED; WCAG 3.0 (draft, ~2029) brings XR into scope — v2 radar	[13]
R-9	godot --check-only parses errors only; warnings need -debug (CI hang risk without timeout)	CONFIRMED; wrap every Godot CLI call in a timeout	[14][15]
R-10	gdUnit4 headless CLI invocation; fresh checkout needs --import first	CONFIRMED	[10][16]
R-11	bubblewrap needs unprivileged user namespaces; can fail EPERM on AppArmor-restricted kernels	CONFIRMED; sandbox preflight + loud fallback mandatory (D-05)	[8][9]
R-12	uv sync --frozen is CI-standard; uv lock --check fails on stale lockfile	CONFIRMED	[22]
R-13	Kimi Code swarm mechanics (Part 5)	CONFIRMED: CLI sessions independent, filesystem coordination; web swarm 100–300 sub-agents	[17][18][21]

3.1 Claims that still need build-time verification

Claim	Why open	Owner
EngineDebugger capture-channel availability across build types (editor-launched / debug export / release export)	v2.0 marked INFERRED; payload behaviour NOT VERIFIED	W-addon, Wave 2
num_ctx 8192 sufficiency for depth-6 scene tree + image tokens	INFERRED risk; measure with the real demo project	W-operator, Wave 2
A1 reachability via OpenXRAPIExtension from GDExtension	INFERRED from API surface; no worked example exists	W-layer, Wave 6 (gated on D-04)
OpenXR-MotionCompensation hooked-function set	BLOCKED in v2.0 (GitHub retrieval); clone the repository to confirm	W-layer, Wave 6

Part 4 — v3.0 findings: seeded suspects, dispositioned

The v3.0 audit seeded sixteen suspects against v2.0. Dispositions below. "Confirmed" here means confirmed by document analysis against v2.0's text; code-level confirmations are re-executed during the build (Wave 0/1) per the executable-claims rule.

Table 4-1 Suspect dispositions S-01...S-16

ID	Suspect	Disposition	Severity	Fix (wave)
S-01	§7.1 mandates digest-pinned Actions; §7.2 workflow pins by tag (@v4, @v5, @v2.3.8)	CONFIRMED — internal contradiction in the supply-chain gate	MAJOR	Pin all actions by commit digest (W1)
S-02	§3.2 "restrictive permissions" on token file vs §6.1 code setting none	CONFIRMED — claim unimplemented; GDScript FileAccess exposes no permission API, so the prose claim was unimplementable as written	MAJOR	Document reality; enforce via OS/user-dir hardening; test what the platform allows (W2)
S-03	§6.1 calls Capture.capture_async but §6.6 declares class_name OperatorCapture; inject.gd has no listing anywhere — A3, the only v1 actuation tier, lacks a reference implementation while M3 requires it	CONFIRMED — name mismatch + coverage gap	MAJOR	Fix class name; write inject.gd as first-class deliverable with binding-assertion tests (W2)
S-04	§6.4 is_symlink() after .resolve() is dead code; TOCTOU window between validation and write	CONFIRMED — dead check; resolve() already eliminates symlinks	MINOR	Remove dead check; document residual TOCTOU; write with O_NOFOLLOW-equivalent discipline where the platform allows (W2)
S-05	addon/openxr_operator/ vs res://addons/openxr_operator/ — install mapping undocumented	CONFIRMED — ABSENT install step	MINOR	Install runbook + addon packaging step (W2/W5)
S-06	num_ctx 8192 vs scene tree + image token budget	INFERRED risk — measure on the real demo project	MINOR	Measure; raise default or bound tree size (W2)
S-07	Debugger-channel availability in release exports; remote-debug endpoint auth posture	OPEN — requires runtime verification (v2.0: INFERRED)	MAJOR	Matrix test across build types; document constraint (W2)

S-08	"Append-only" audit log with no integrity mechanism	CONFIRMED — tamper-evidence ABSENT	MAJOR	Hash-chain records (each record includes SHA-256 of predecessor) + verification tool (W4)
S-09	Model weights absent from supply chain: no registry provenance, no digest pinning	CONFIRMED — ABSENT from §7.5	MAJOR	ollama pull model@sha256:... pinned in config; digest in SBOM; CI check (W1/W4)
S-10	CI downloads gitleaks binary without checksum verification	CONFIRMED — inside the pipeline that enforces supply-chain discipline	MAJOR	Pin version + verify SHA-256 of every downloaded binary (W1)
S-11	No minimum-hardware / latency envelope for engine + capture + 7B–12B Q4 VLM	CONFIRMED — ABSENT	MAJOR	Preflight measures and records the envelope; README documents the floor (W0/W4)
S-12	gdUnit4 "v6.x" permits versions incompatible with Godot 4.7	CONFIRMED by research — 4.7/4.7.1 requires $\geq$ v6.2.0 ^[10]	MAJOR	Pin gdUnit4 $\geq$ 6.2.0 in register and CI (W1)
S-13	Qwen3-VL caution note cites a March 2026 issue	STALE — fixes recorded in downstream 2026.4.x releases ^[7] ; keep "verify your exact tag"	NIT	Re-date the note (W1)
S-14	Approval-gate fatigue: no rate bounds, no batching, no rubber-stamp detection	CONFIRMED — ABSENT policy	MINOR	Approval UI batches related diffs; audit log records approval latency for review (W4/W5)
S-15	Settle detector: timeout honouring; invalid on_timeout values silently fall through to proceed	CONFIRMED (minor) — invalid values coerce to TIMEOUT_PROCEED	MINOR	Validate on_timeout at flow-parse time; timeout test mandatory (Maestro shipped this bug — v2.0 §5.1) (W3)
S-16	§6.1: constant-time compare early-returns on length; rate-window boundary; auth deadline from accept not handshake	CONFIRMED — all three are nits, none exploitable at the stated limits	NIT	Document; optional hardening (W2)

Disposition summary: 12 confirmed findings (8 MAJOR, 3 MINOR, 1 NIT), 2 stale/nit documentation updates, 1 inferred risk to measure, 1 open item requiring runtime verification. None challenges the architecture. All are scheduled in the Part 6 wave plan.

Part 5 — The Kimi Code swarm operating model

5.1 Facts the model is built on

- A Kimi Code CLI session is an independent agent: its own context, its own transcript stored as plain files under `~/.kimi-code/sessions/`, resumable with `kimi --continue`, nameable with `/title`. Two sessions know nothing about each other^[17].
- There is no native swarm mode in the CLI (open feature requests). A CLI swarm is a topology of sessions coordinated through the filesystem^{[17][18]}.
- Inside one session, Kimi Code ships three built-in sub-agents — coder, explore, plan — which parallelize steps of one task^[17].
- Kimi's web Agent Swarm (kimi.com/agent-swarm) self-orchestrates 100 sub-agents / 1,500 tool calls on K2.5 and 300 sub-agents / 4,000 steps on K2.6 and K3^{[19][21][20]}.

Consequence: coordination state must live in the repository, not in any agent's context. Every agent reads its task from a file, writes its status to a file, and never relies on telling another agent anything.

5.2 Topology

Table 5-1 Swarm roles (one Kimi Code session each)

Role	Session title	Owns (write access)	Reads
Orchestrator	oxr-orchestrator	Root files, .github/, tools/, docs/, contracts (protocol.gd, driver.py, schema.py, flow schema), swarm/, all ledgers	Everything
Worker: engine	oxr-w-addon	addon/, demo_project/	Contracts, its task cards, kit Part 2
Worker: agent core	oxr-w-operator	operator/src/openxr_operator/ (except driver.py, schema.py), operator/tests/	Contracts, its task cards
Worker: UI	oxr-w-ui	operator/src/openxr_operator/ui/	Contracts, its task cards
Worker: layer	oxr-w-layer	layer/	Contracts, its task cards, v2.0 §2.3/ §6.8
Worker: QA	oxr-w-qa	flows/, VERIFICATION.md, runs/	Everything; writes no production code

Ownership is exclusive: a file has exactly one writer. Cross-cutting files (contracts, CI, licences) belong to the orchestrator alone. Workers request contract changes by writing a proposal to `swarm/requests/`; the orchestrator applies or rejects. This is the single rule that prevents swarm merge chaos.

5.3 Coordination protocol (files, not messages)

```

swarm/
  STATUS.md          # one line per role: state, current task, timestamp – orchestrator
maintains
  requests/          # worker → orchestrator contract-change proposals
  decisions/          # orchestrator rulings on requests, D-## cross-referenced
  tasks/
    W1-addon-01.md    # task card: scope, files owned, acceptance, evidence required
    ...
  AUDIT.md  PLAN.md  VERIFICATION.md  BLOCKERS.md  SHIP.md  # ledgers (orchestrator-owned)

```

Task-card format (every card, no exceptions):

```

id: W2-operator-03
role: oxr-w-operator
owns: [operator/src/openxr_operator/settle.py, operator/tests/test_settle.py]
reads: [docs/contracts/flow-schema.json, swarm/tasks/_context.md]
do: implement the settle detector per kit $5.2 with on_timeout validation (S-15)
acceptance:
  - pytest passes: timeout honoured to ±50ms; invalid on_timeout rejected at parse
  - mypy --strict clean; ruff clean
evidence: paste command outputs into the task card's Evidence section
done_when: all acceptance items executed, evidence attached, card moved to swarm/done/

```

5.4 Failure and escalation rules

Situation	Rule
Worker blocked on another role's file	Write a request to swarm/requests/, mark the task blocked, pick the next card. Never edit outside ownership.
Same failure survives 3 fix attempts	Stop; write BLOCKERS.md entry with root-cause analysis. No thrashing.
Contract change needed	Orchestrator applies it, bumps protocol version if wire-visible, notifies via STATUS.md; affected workers re-read contracts before resuming.
Rule 9 trigger (destructive/irreversible)	All agents halt that action and escalate to the human. No exceptions.
Security-gate change	Human approval required, always. This cannot be delegated to any agent.
Two workers need the same new helper	It goes in the contract layer, owned by the orchestrator.

5.5 Scaling down and up

Single session (minimum viable). One Kimi Code session runs Appendix B verbatim, using its plan/explore/coder sub-agents for parallelizable steps. Waves become sequential phases. Everything else — ledgers, gates, stop points — is unchanged.

Web Agent Swarm (maximum). On kimi.com/agent-swarm, paste the orchestrator prompt (P-0) with the wave plan; the K2.6/K3 orchestrator self-decomposes across sub-agents^[21]. Keep the ownership matrix and task-card format in the prompt — the learned orchestrator parallelizes well, but exclusive file ownership is a discipline the prompt must impose, because sub-agents share one workspace.

Part 6 — Build plan: waves, task cards, acceptance criteria

Waves are dependency-ordered; within a wave, cards run in parallel across roles. Waves map to v2.0 milestones M0–M7. RPI applies inside every card. Human stop points: starting W6 (D-04), any security-gate change, any Rule 9 action, adopting a model not on the pass-list, shipping with any D-decision on an unexamined default.

Table 6-1 Wave plan

Wave	Milestone	Cards (role → scope)	Wave exit gate
W0 — Preflight & ingest (sequential, orchestrator)	—	Orchestrator: toolchain preflight (Godot 4.7.1, Python 3.11+, uv, CMake, Ollama); bubblewrap smoke test with EPERM/AppArmor handling (R-11, D-05); GPU/VRAM/RAM inventory vs model table (S-11); dry-run every CI command incl. timeout-wrapped Godot CLI (R-9/R-10); ingest kit + ledgers; confirm D-01...D-08 answered or defaults explicit; write AUDIT.md skeleton.	Preflight report on disk; every missing/wrong tool is a P0; D-04/D-05 answered by the human.
W1 — Repo, contracts & gates (orchestrator only)	M0 + S-01, S-09, S-10, S-12, S-13	Orchestrator: repo layout; split licences; uv project with committed lockfile; freeze the five contracts (wire protocol v2, Driver, action schema, flow schema, ModelBackend) under docs/contracts/ and operator/; all CI gates wired; Actions pinned by commit digest; gitleaks + all downloaded binaries checksum-verified; gdUnit4 ≥ 6.2.0; model digest pinning scaffold; re-date Qwen3-VL note.	CI green on an empty codebase; vendor gate demonstrably fails when openai is added to the lockfile and passes when removed; tag-pinned Action demonstrably fails the new digest gate.

W2 — Planes behind contracts (parallel)	M1, M2, M3 + S-02... S-07, S-16	oxr-w-addon: authenticated server (token lifecycle documented per S-02), scene tree with depth/filter, T2 mirror capture with frame_post_draw gating + black-frame detection, T3 desktop mode, inject.gd as a first-class deliverable with binding assertion (S-03), editor bridge, debugger-availability matrix (S-07), class-name fix (S-03). oxr-w-operator: Python client, fsguard with dead-check removal (S-04), settle detector with on_timeout validation (S-15), schema validation, context-budget measurement (S-06). oxr-w-qa: contract tests (fake engine ↔ fake agent), auth rejection tests, path-traversal tests, demo project skeleton.	Unauthenticated command closes the connection; wrong protocol version reports clearly; 1 MiB frame rejected; 51st msg/s closes the peer; capture returns a non-black frame at a declared tier against an XR-enabled demo; bogus tracker name raises instead of no-op; path traversal rejected and logged.
W3 — Flow runner (parallel, contracts frozen)	M4	oxr-w-operator: selector grammar + resolution logging, YAML flow parser, runner, artifact bundle, JUnit emitter. oxr-w-qa: flows/ library incl. smoke flow; settle timeout tests; run-bundle completeness checks.	Demo flow runs headless in CI with the model disabled and produces a complete run bundle; no sleep anywhere in the runner; timeout honoured (executed, ±50ms).
W4 — Agent loop (parallel)	M5 + S-08, S-09, S-11, S-14	oxr-w-operator: ModelBackend reference adapters — Ollama and llama.cpp shipped, digest-pinned model (D-10); the eight hosted adapters registered-but-unshipped pending per-provider endpoint verification (D-12); permanent excluded-vendor denylist enforced centrally in resolve() (V-09/D-11); no automatic fallback — unreachable means a named error (D-13); schema-constrained decode + validate + one retry, capability allowlist, diff-then-approve, hash-chained audit log (S-08), egress restriction + runtime verification, filesystem jail with preflight. oxr-w-qa: security spot-check suite executed every loop: unby-passable-approval test, traversal test, egress test, tamper-evidence verification tool.	A mutating action cannot execute without an approval record (attempted, logged); schema-violating model output rejected, retried once, aborted; sandbox unavailable ⇒ mutating flows refuse loudly; resource envelope measured and in README.
W5 — Web UI & install (parallel)	M6 + S-05, S-14	oxr-w-ui: loopback bind, Origin allowlist, token gate, vendored assets, CSP, loading/empty/error states, keyboard navigation, visible focus, approval UI with diff batching. oxr-w-addon: addon packaging + install runbook (S-05). oxr-w-qa: WCAG 2.2 AA baseline pass; network-unplugged test.	No third-party origin in any response; page functions with the network cable unplugged; WCAG baseline pass; install runbook executed from scratch on a clean profile.

W6 — Layer (T1/A2) (gated on D-04, human stop)	M7 oxr-w-layer: explicit layer, Vulkan swapchain capture, layer pose injection, Monado simulated-device CI job, MotionCompensation hooked-function confirmation (clone, don't infer). oxr-w-qa: T1-vs-T2 frame divergence test; implicit-layer gate stays green.	A layer-captured frame differs measurably from the T2 mirror under stereo rendering; no implicit manifest anywhere; loader activation is per-process only.
--	--	--

6.1 Verification loop (every wave, QA-led)

1. Build from clean state.
2. Run every CI gate locally.
3. Run every flow in flows/ model-disabled, then model-enabled; record per-step capture tier, injection tier, and settle outcome in VERIFICATION.md. A run whose steps are all TIMEOUT_PROCEED is not synchronising — say so.
4. Execute the security spot-checks: unauthenticated connection closes; mutating action without approval fails; traversal rejected; bad Origin rejected; `grep -rn "0\.\0\.\0\.\0" operator/ addon/ empty`.
5. Fix failures. Same failure three times → BLOCKERS.md, stop.

Exit condition: full DoD (Part 8) green on two consecutive full runs — three for flow/smoke and XR-timing surfaces, which are flaky-async by nature.

Part 7 — Agent prompts (orchestrator and workers)

Seven prompts, all self-contained. Paste P-0 into the orchestrator session first; it writes the task cards and STATUS.md. Paste each worker prompt into its own titled session (/title oxr-w-addon etc.) in the same repository. For a single-session build, skip this Part and use Appendix B.

P-0 — Orchestrator

```
# ROLE
You are the orchestrator of the OpenXR Operator build swarm. You coordinate; you also
own all cross-cutting files. You are the only agent that writes: repository root files,
.github/, tools/, docs/, the five contracts (protocol.gd, driver.py, schema.py, the flow
JSON Schema, model.py), swarm/, and the ledgers (AUDIT.md, PLAN.md, VERIFICATION.md,
BLOCKERS.md,
SHIP.md).
```

```
# CANONICAL INPUTS (in this order of authority)
1. docs/BUILD_KIT_v3.pdf – architecture, tier model, security design, wave plan.
2. Audit ledgers – findings F-###, amendments A-##, suspects S-01..S-16 dispositions.
3. The repo as it exists. Actual behaviour outranks both documents; log disagreements
   in AUDIT.md.
```

```
# STANDING RULES
RPI; Rule 9 (no destructive/irreversible action without explicit human approval); zero
fabrication with the five labels (CONFIRMED/INFERRED/BLOCKED/NOT ATTEMPTED/UNTESTED);
executable claims must be executed; licence split invariant (addon+layer MIT, operator
AGPL-3.0-or-later); vendor gate (no Meta/OpenAI/xAI, Google permitted, vendorscan.py
stays green); security gates invariant (loopback-only, explicit layer only, no browser
path to engine, no mutation without approved diff); osv-scanner never Trivy; Actions
pinned by commit digest; downloaded binaries checksum-verified; model pinned by digest;
uv sync --frozen. Terse output.
```

```
# WORK
1. Run Wave W0 (preflight & ingest) yourself: toolchain check, bubblewrap smoke test
   (EPerm ⇒ escalate D-05 to the human with evidence), hardware envelope, CI dry-run
   with timeout-wrapped Godot CLI. Get human answers for D-04 and D-05 before planning
   beyond W1.
2. Write PLAN.md mapping kit F-001..F-030, S-01..S-16, and amendments to wave tasks.
3. Execute W1 yourself: repo layout, licences, contracts frozen under docs/contracts/,
   all CI gates green on an empty codebase.
4. For each later wave: write task cards to swarm/tasks/ (format in kit §5.3), update
   swarm/STATUS.md, and wait for cards in swarm/done/. Review every done card: evidence
   present, gates green, ownership respected. Reject incomplete cards with reasons.
5. Rule on swarm/requests/ promptly; record rulings in swarm/decisions/.
6. After each wave: run the verification loop (kit §6.1) with the QA worker's results;
   integrate; commit.
7. Human stop points: W6 start (D-04), any security-gate change, any Rule 9 action,
   any unlisted model, any D-decision still on an unexamined default at ship time.
8. Finish with SHIP.md per Appendix B, including F-###/S-##/A-## disposition tables and
   the measured resource envelope.
```

P-1 — Worker: engine (addon)

ROLE

You are oxr-w-addon. You own addon/ and demo_project/ exclusively. You never write outside them. Contract changes: propose in swarm/requests/, never edit contracts.

TASKS

Read your cards from swarm/tasks/W*-addon-*.md. Work them in id order. For each: RPI (research the card's references, plan in the card, then implement), one logical change per commit referencing the card id and finding id, then move the card to swarm/done/ with executed evidence pasted in.

NON-NEGOTIABLES

- GDScript fully typed; gdlint/gdformat clean; godot --headless --check-only --script passes on every file (timeout-wrapped).
- Loopback 127.0.0.1 only; token-first auth; protocol version in the auth frame; limits per kit §3.2. Token-file permissions: implement the most restrictive the platform allows and make docs match the code (S-02) – never claim what the code doesn't do.
- Capture: T2 mirror viewport, await RenderingServer.frame_post_draw before readback, black-frame detection, tier recorded on every frame. Never return a silent black screenshot.
- Injection: tracker names constrained to left_hand/right_hand, pose default, and a post-injection assertion that an XRController3D bound. Silent no-op is a failure.
- Editor-side work goes over the EngineDebugger capture channel, never EditorInterface from the game process. Verify and document debugger-channel availability across build types (S-07).
- Class names must match what other files reference (S-03).

Escalate via swarm/requests/ and mark the card blocked when you need anything outside your ownership. Three failed fix attempts on the same issue → write BLOCKERS.md entry, stop that card.

P-2 — Worker: agent core (operator)

```
# ROLE
You are oxr-w-operator. You own operator/src/openxr_operator/ EXCEPT driver.py,
schema.py and model.py (orchestrator-owned contracts), plus operator/tests/. Read contracts
from
docs/contracts/; propose changes via swarm/requests/.

# TASKS
Cards swarm/tasks/W*-operator-*.md, in id order, RPI per card, one logical change per
commit, evidence executed and pasted before moving to swarm/done/.

# NON-NEGOTIABLES
- mypy --strict and ruff clean; pytest green.
- Model output is data: schema-validate, capability-allowlist, diff-then-approve for
every mutating action. No auto-apply path exists. Not behind a flag.
- Path containment per kit $6.4 minus the dead symlink check (S-04); document the
residual TOCTOU.
- Settle detector honours its timeout (tested, ±50ms) and rejects invalid on_timeout
at flow-parse time (S-15). No sleep anywhere in the runner.
- Audit log is hash-chained tamper-evident, written before execution (S-08), with a
verification tool.
- Model access goes through ModelBackend only, resolved via backends.resolve()
(D-10). Local first, not local only (D-12): hosted adapters stay
registered-but-unshipped until per-provider endpoint verification; the
permanent denylist is enforced centrally in resolve(), never delegated to
adapters (V-09). No automatic fallback (D-13): unreachable means a named
error. Adapters declare their constrained-decoding mode; anything below
native degrades to validate-and-retry and the run report must say so. Plain
httpx, no vendor SDK (D-11: the wire format is not an artifact). Model pinned
by digest (S-09), temperature 0, one retry then abort. Measure num_ctx
sufficiency against the real demo scene tree and record the number (S-06).
- The runner passes its whole suite with the model disabled. The model is a client,
not a component.
- Sandbox preflight at CLI startup; unavailable ⇒ mutating flows refuse loudly (D-05).
Same escalation and BLOCKERS rules as every worker.
```

P-3 — Worker: UI

```
# ROLE
You are oxr-w-ui. You own operator/src/openxr_operator/ui/ exclusively.
# NON-NEGOTIABLES
- Loopback bind only; Origin validated against an exact allowlist; token-gated.
- Zero third-party origins: all assets vendored; CSP set; page fully functional with
  the network cable unplugged (test it that way).
- Loading, empty, and error states on every async surface; keyboard-navigable; visible
  focus; semantic markup; WCAG 2.2 AA baseline.
- Approval surface: unified diff, approve/reject/edit, related diffs batched (S-14);
  rejections and their diffs go to the audit log.
- The browser never opens a socket to the engine port. All engine traffic goes through
  the Python plane.
Cards from swarm/tasks/W*-ui-*.md; RPI; executed evidence; BLOCKERS after three failed
attempts on one issue.
```

P-4 — Worker: layer (gated on D-04)

```
# ROLE
You are oxr-w-layer. You own layer/ exclusively. You start only when the human has
approved D-04 and the orchestrator opens your cards.
# NON-NEGOTIABLES
- The layer is explicit and environment-scoped only: XR_API_LAYER_PATH +
  XR_ENABLE_API_LAYERS per process. Never write an implicit.d manifest, never touch
  registry keys, no installer, nothing persistent. A CI gate greps for this and must
  stay green.
- C++17, -Wall -Wextra -Werror; CMake per kit §6.8; Vulkan swapchain handling first.
- Clone OpenXR-MotionCompensation and confirm its hooked-function set from source –
  do not implement from inference (v2.0 was BLOCKED on this).
- Budget real engineering for the swapchain interop; the working references are
  GFXReconstruct, OpenXR-Vk-D3D12, and OpenXR-Layer-OBSMirror (kit §2.3).
- Acceptance: a layer-captured frame differs measurably from the T2 mirror under
  stereo rendering, and the run report records tier T1 honestly.
```


P-5 — Worker: QA

ROLE

You are oxr-w-qa. You own flows/, VERIFICATION.md, and runs/. You write no production code. Your job is to make every other role's claims executable.

WORK

- Contract tests: fake engine ↔ fake agent round-trip; auth rejection; wrong protocol version; oversize frame; rate limit; traversal rejected AND logged; unbyypassable approval attempted; bad Origin rejected.
- The flows/ library, starting with flows/smoke.yaml; run headless in CI with the model disabled (T3).
- Every verification loop: clean build, all gates, all flows model-disabled then model-enabled; record per-step capture tier, injection tier, and settle outcome in VERIFICATION.md. Flag any all-TIMEOUT_PROCEED run as not-synchronising.
- Security spot-checks every loop, executed not read.
- Two consecutive clean full runs (three for flow/XR-timing surfaces) before any wave is called done. Three failed fix attempts anywhere → BLOCKERS.md.

Part 8 — Definition of done, decision register, open items

8.1 Definition of done (contract)

- Fresh clone builds and runs with the documented commands on the stated minimum hardware; the measured resource envelope (GPU/VRAM/RAM floor, per-step latency) is in the README and was measured, not estimated.
- Zero lint, type, parse, or compiler-warning errors, or documented suppressions with justification. Godot CLI checks run under a timeout; no CI step can hang in the debugger.
- Every flow in flows/ completes end to end with no unhandled exception and no engine debugger errors, on two consecutive full runs from clean state (three for flow/XR-timing), with per-step tier and settle evidence in the run bundle.
- The approval gate is proven unbyypassable by execution; attempts are logged.
- Audit log is hash-chained tamper-evident with a verification tool; "append-only" as intention does not count.
- Model weights pinned by digest and in the SBOM; all Actions pinned by commit digest; all CI-downloaded binaries checksum-verified.
- Sandbox preflight runs at CLI startup; unavailable ⇒ mutating flows refuse loudly.
- osv-scanner clean or accepted risks recorded with reason and expiry; full git history scanned for secrets; token-file permissions implemented and documented honestly.
- Web UI: loopback, Origin allowlist, token gate, vendored assets, CSP, states, keyboard, focus, WCAG 2.2 AA baseline — verified with the network unplugged.
- Vendor gate and licence gate pass; SBOM generated and committed for the release.
- README (setup, build, run, tier model, hardware floor, troubleshooting); CHANGELOG; SHIP.md with disposition tables (F-001...F-030, S-01...S-16, A-##) where every claim is verified-by-execution or marked UNTESTED.

8.2 Decision register (human-owned; none assumed)

ID	Decision	Default	Blocks
D-01	Primary platform for the API layer	Linux first, Windows second	W6
D-02	Accept Meta funding proximity in the Godot XR toolchain	Accept, documented	—
D-03	In-engine addon licence	MIT	W1
D-04	Build the API layer in v1 or defer	Defer; ship T2/T3 in v1	W6 (human stop)

D-05	Agent sandbox mechanism	bubblewrap on Linux, container elsewhere	W0 (preflight may force it)
D-06	Destructive actions ever auto-apply	Never	W4
D-07	Reuse an existing Godot MCP server vs build the bridge	Build, after licence audit of candidates	W2
D-08	Target the OMBI Scene Object Model in selectors	No — no normative spec exists	W3
D-09	Godot 4.8 adoption policy (new in v3.0)	Stay on 4.7.1 for v1; evaluate 4.8 at v2	W0
D-10	Model backend policy (added 9 Aug 2026 — fixes v2.0's concrete OllamaClient with no seam)	ANSWERED, amended after v2.1 audit: protocol + twelve registered adapters (4 local, 8 hosted); Ollama + llama.cpp shipped; denylist applied centrally in resolve(), not per-adapter (V-09)	W4
D-11	Vendor gate scope: artifacts vs OpenAI-compatible wire format	ANSWERED: artifacts only; the JSON shape over plain httpx carries no OpenAI code	—
D-12	Inference locality (added 9 Aug 2026 — resolves v2.1's V-01/V-02 self-contradiction)	ANSWERED by the human (option B): local first, not local only — hosted providers opt-in, one egress domain each, never an excluded vendor; hosted adapters registered-but-unshipped	W4
D-13	Backend fallback behaviour	ANSWERED by the human (option B): manual configuration only, no automatic fallback; unreachable backend fails with a named error	W4

8.3 Open items carried forward (from v2.0 Part 10, updated)

- OpenXR-MotionCompensation hooked-function set — clone and confirm (W6).
- FSF AGPL FAQ exact wording on §13 and plug-ins as combined works — verify before any legal notice quotes it.
- OMBI Scene Object Model normative specification — site is a client-rendered SPA; revisit at v2 (D-08).
- Licence, stars, and activity of the five candidate Godot MCP servers — gates D-07.
- Exact licence strings on the Ollama gemma3 and moondream model cards — verify at model pinning (W4).
- GFXReconstruct issue #2145 (known-working OpenXR applications) — W6 background.

8.4 Non-goals for v1

Unity or Unreal. Cloud inference of any kind. Multi-agent orchestration of the shipped agent (the swarm builds the product; it is not the product). Voice input. Model fine-tuning. Godot Asset Store publishing. The OMBI Scene Object Model (D-08). Any feature of v1.0's "Extending the Kit" section.

Appendix A — Adversarial QA audit prompt v3.0 (full text)

Paste verbatim as the first message in an audit project whose knowledge base contains the artifact under audit. Phase-gated; reply GO to advance.

```
# Adversarial QA & Production-Readiness Audit – OpenXR Operator (v3.0)
```

```
**What this file is.** A hardened, domain-specialized successor to the generic *Adversarial  
QA & Production-Readiness Audit – [PROJECT]* template, adapted to the *OpenXR Operator Build  
Kit v2.0* (8 August 2026). It supersedes both the generic template and the kit's own §9.1  
prompt. Section 1 documents what changed and why. Section 2 is the research baseline,  
verified 9 August 2026. Section 3 is the paste-ready audit prompt – self-contained, phase-  
gated, runnable as the first message in an audit project whose knowledge base contains the  
build kit.
```

```
---
```

```
## 1. Adaptation analysis – what v3.0 changes and why
```

```
### 1.1 Holes in the generic template (fixed)
```

Hole	Fix in v3.0
No mechanism to force **execution** of claims – an auditor can "verify" code by reading it. The kit itself contains evidence this fails (F-028: a typo in a documented failure mode suggests v1.0 code was never run).	New standing rule: ***executable claims must be executed.*** Every Part 6 listing must be compiled, parsed, or run, not read. A claim verified by reading is labelled INFERRED, never CONFIRMED.
Epistemic labels (VERIFIED/INFERRED/ASSUMED/UNKNOWN) are weaker than the kit's four-state model and allow "I didn't look" to masquerade as "unverifiable."	Adopts the kit's four states – CONFIRMED / INFERRED / BLOCKED / NOT ATTEMPTED – and adds the §9.1 rule that presenting NOT ATTEMPTED as BLOCKED is itself a finding against the auditor.
No stale-claim discipline. A KB dated "today" decays from tomorrow.	Adds a ***claim re-verification schedule*** (§3, Phase 1): every load-bearing dated claim in the kit is enumerated and must be re-verified against primary sources as of audit day.
Phase 6 rewrites a kickoff prompt but never audits the **reference implementation** as code.	Dedicated ***Phase 4 – Reference implementation audit***: prose-vs-code drift between Parts 2–5/7 and Part 6/§7.2 is its own finding class.
No treatment of a KB that carries its **own** prior audit (the kit contains a full F-### ledger, decision register, and open-items list). Auditors either rubber-stamp it or ignore it.	New rules: the kit's findings carry no authority – each must be re-derived or explicitly re-confirmed; the Part 10 BLOCKED and NOT-ATTEMPTED lists become ***mandatory closure items***; the D-01..D-08 decision register must be re-validated, not inherited.
No seeded adversarial targets, so audit quality depends entirely on auditor luck.	Adds a ***seeded suspects annex*** (verify-or-clear list). Each suspect is a location plus a question, not a conclusion: the auditor must confirm, refute, or re-severity each one with evidence. Seeding suspects without verdicts preserves independence while removing blind spots.
UX phase ignores that this product's "first five minutes" includes hardware and model-download prerequisites.	Phase 5 audits the full onboarding path: GPU/VRAM envelope, Ollama install, model pull size, Godot version match, addon install path – as UX, not just ops.
Verdict phase has no self-audit.	Phase 8 adds an ***audit-the-auditor*** pass: list claims you verified by reading rather than execution, retrievals you skipped, and suspects you cleared on low confidence.

1.2 Holes in the kit's own §9.1 prompt (fixed)

§9.1 was a good compression of the generic template but lost load-bearing parts and missed kit-specific attack surface:

1. ***It dropped the per-phase "Output:" contracts*** and the "What good looks like" closing section. Restored.
2. ***It dropped the comparables scan's dead-projects requirement and the checklist delta.*** Restored, with named comparables (Maestro, OpenXR-Toolkit, OpenXR-MotionCompensation, GFXReconstruct, candidate Godot MCP servers).
3. ***It never audits Part 6 as code.*** §9.1's Phase 1 verifies APIs exist; it never requires compiling the listings. Fixed via Phase 4.
4. ***It never closes the kit's own open items.*** Part 10 lists 5 BLOCKED and 15 NOT-ATTEMPTED items; §9.1 mentions none of them. Fixed in Phase 1.
5. ***It ignores the decision register.*** D-01..D-08 defaults are load-bearing (e.g., D-04 defers the API layer, which determines whether F-010 is ever fully resolved). Fixed in Phase 2.
6. ***It omits kit-specific threat surface:*** the debugger channel's availability and authentication posture, token-file permissions, audit-log tamper evidence, model-weight supply chain (Ollama registry provenance/digests), approval-gate fatigue, and egress enforcement being outside the process it constrains. Added to Phase 3.
7. ***It omits the resource envelope.*** A 7B–12B quantized VLM plus a running XR engine plus a capture pipeline has a hardware floor the kit never states. Added to Phases 3 and 5.

1.3 New findings seeded by v3.0's own analysis (the verify-or-clear annex)

These were produced by adversarial reading of the kit during adaptation. They are ****suspects, not verdicts**** – the auditor must confirm, refute, or re-severity each with evidence. Full list in the prompt (§3, Annex A); the highest-value ones:

- ****S-01**** – §7.1 mandates pinning every GitHub Action by commit digest ("the Trivy tag-hijack is the reason"); the §7.2 workflow pins everything by `*tag*` (``actions/checkout@v4``, ``astral-sh/setup-uv@v5``, ``osv-scanner-reusable.yml@v2.3.8``). Internal contradiction in the supply-chain gate itself.
- ****S-02**** – §3.2 says the auth token is written "with restrictive permissions"; the §6.1 code sets none, and GDScript's ``FileAccess`` exposes no permission API. Claim may be unimplementable as written.
- ****S-03**** – §6.1 dispatches to ``Capture.capture_async`` and ``Inject.apply``; §6.6 declares ``class_name OperatorCapture`` (name mismatch), and ****`inject.gd`** has no listing anywhere in Part 6 – the only actuation tier shipping in v1 (A3) lacks a reference implementation, while M3's acceptance criteria require it.
- ****S-04**** – §6.4's ``target.is_symlink()`` check runs ****`after`** ``resolve()``, which has already eliminated symlinks: the check is dead code, and a TOCTOU window exists between validation and write.
- ****S-10**** – The CI workflow downloads the gitleaks binary over HTTPS with ****no checksum verification****, in the same pipeline that exists to enforce supply-chain discipline.
- ****S-11**** – No minimum-hardware specification anywhere in the kit: VRAM/RAM floor for Qwen2.5-VL-7B/Gemma 3 12B at Q4, per-step latency budget, or behavior when the model host and the XR runtime contend for the same GPU.

2. Research baseline (verified 9 August 2026)

These results re-verify the kit's most load-bearing time-sensitive claims and are the starting state for the audit's stale-claim sweep. The auditor re-verifies each one again on audit day.

```
| # | Claim in kit | Status 9 Aug 2026 | Evidence |
|---|---|---|---|
| R-1 | Godot issue #100004 (viewport capture black under `use_xr`) is open, unfixed in 4.7
| **CONFIRMED open.** Filed 3–4 Dec 2024, reproducible since 4.1.3, affects Vulkan and
OpenGL; no merged fix located. | github.com/godotengine/godot/issues/100004 [^1^] |
| R-2 | "Godot 4.7.x" is current stable | **STALE-adjacent.** 4.7-stable shipped 18 Jun 2026
("Lights, Camera, Action!" / Director's Cut); **4.7.1 maintenance shipped 14 Jul 2026**; 4.8
is in development. Kit should pin 4.7.1 and state a 4.8 policy. | godotengine.org release
blog and archive [^2^][^3^] |
| R-3 | Chrome 147 enforces Local Network Access on WebSockets (~Apr 2026); Firefox/Safari
do not | **CONFIRMED.** Chromium 147+ requires a permission prompt for public-
origin-loopback/private WebSockets; Firefox and Safari do not enforce. LNA remains
unavailable as a dependable control – kit's conclusion holds. | Chrome LNA WebSocket
guidance [^4^] |
| R-4 | Ollama `format` accepts JSON Schema, composes with `images`, works with vision
models | **CONFIRMED** with one addition: structured outputs are **not supported on
Ollama's cloud tier** – irrelevant to local-only use but must be stated if cloud fallback is
ever proposed (it is a non-goal). | docs.ollama.com/capabilities/structured-outputs [^5^] |
| R-5 | Qwen3-VL has vision-detection integration issues (issue #50569, Mar 2026); verify
before adopting | **PARTIALLY STALE.** Qwen3-VL shipped on Ollama (cloud Oct 2025, local
after) with strong GUI/grounding capabilities; the referenced detection issue records fixes
in downstream releases (2026.4.x). The kit's caution note needs re-dating; "verify your
exact tag" remains correct advice. | ollama.com/blog/qwen3-vl; openclaw issue #50569 [^6^]
[^7^] |
| R-6 | XR_EXT_conformance_automation: Monado implements it, most consumer runtimes don't |
**CONFIRMED and strengthened.** Monado 25.1.0 (10 Dec 2025) and the Khronos-funded Monado
Upgrade Project (2026) include conformance-automation support – AI's Linux path is real and
improving. | Collabora Monado 25.1.0; vr.org Monado report [^8^][^9^] |
| R-7 | gdUnit4 "v6.x" in the dependency register | **IMPRECISE – finding.** gdUnit4
**v6.1.x supports only up to Godot 4.6.3; Godot 4.7/4.7.1 requires ≥ v6.2.0** (v6.2.1
current). The register's "v6.x" pin permits a version incompatible with the target engine. |
github.com/godot-gdunit-labs/gdUnit4 compatibility table [^10^] |
| R-8 | WCAG 2.2 AA is the right accessibility baseline | **CONFIRMED.** WCAG 2.2 remains
the operative W3C Recommendation; WCAG 3.0 is a Working Draft (Mar 2026) not expected before
~2029 – but 3.0 explicitly brings XR into scope, so XAUR/XR accessibility belongs on the v2
radar. | WCAG 3.0 status reporting [^11^][^12^] |
```

3. The audit prompt (paste-ready)

Everything below the rule is the deliverable prompt. Paste it as the first message in a project whose knowledge base contains the OpenXR Operator Build Kit v2.0 and any successor artifacts.

Adversarial QA & Production-Readiness Audit – OpenXR Operator (v3.0)

****How to run:**** Paste as the first message inside the project whose knowledge base holds the OpenXR Operator Build Kit v2.0 (and any later artifacts). The audit is phase-gated: run one phase per response and stop. Reply GO to advance, GO n to jump to phase n, or give corrections before advancing. Annexes A and B are part of the prompt.

****PROJECT:**** OpenXR Operator – local-first AI agent (v3.1, D-12) that observes and drives Godot XR applications
****INTENT:**** An agent observes a running Godot XR app, reads its scene tree, and acts on it via a declarative flow runner, using a locally hosted vision-language model; ships to XR developers as a Godot addon + Python CLI/UI + optional OpenXR API layer
****DEPLOY TARGET:**** Developer workstation. No hosted component required (v3.1: hosted model providers are opt-in by manual configuration, single egress domain, never an excluded vendor – D-12/D-13). addon/ (MIT), layer/ (MIT), operator/ (AGPL-3.0-or-later)
****OUT OF SCOPE:**** Unity/Unreal, cloud inference, multi-agent orchestration, the OMBI Scene Object Model (kit D-08)
****ARTIFACT UNDER AUDIT:**** OpenXR Operator Build Kit v2.0, 8 August 2026, and any KB documents it supersedes or cites

Mission

You are the adversarial QA auditor for this project. The build kit is the artifact under audit – treat every Part as a set of claims to be tested, not as context to be trusted.
****The kit contains its own prior audit (F-001...F-030, decisions D-01...D-08, open items in Part 10). None of it carries authority.**** Re-derive or explicitly re-confirm every finding you rely on; a prior finding you adopt without independent evidence is marked as such. Your job: find every reason this project fails before a line of production code is written, close the kit's own open items, then produce the numbered amendments and the corrected implementation kickoff prompt that take it to production.

Deliverables, in order: a findings ledger, numbered amendments, a rewritten kickoff prompt, and a go/no-go verdict – each defensible line by line.

Posture

Audit, don't assist. Improve the spec, not morale.
Burden of proof is on the kit. An unsupported claim is a finding, not background – including claims in the kit's own "CONFIRMED" ledger entries.
A positive verdict carries the same evidentiary burden as a teardown. "Looks solid" is not a finding; show the closest calls you cleared.
Disagree with the kit and with me when warranted. If the project as specified should not be built, say so and state why.

Operating rules

1. ****Zero fabrication.**** Every external factual claim gets a primary source with a date, or an explicit label. Never fill gaps with plausible detail.
2. ****Four epistemic states, used honestly:**** CONFIRMED (primary source, cited, dated), INFERRED (reasoning shown), BLOCKED (what you tried, how it failed), NOT ATTEMPTED (say so plainly). Presenting NOT ATTEMPTED as BLOCKED is a finding against you.
3. ****Executable claims must be executed.**** The kit claims every Part 6 listing is "complete and runnable." Test that claim: parse or compile every GDScript, Python, C++, CMake, YAML, and JSON listing with the actual toolchain (godot --headless --check-only, python -m py_compile or mypy, a C++ compiler, yamllint). A claim verified only by reading is INFERRED, never CONFIRMED. Where execution is impossible in this environment, mark NOT ATTEMPTED and enumerate what a human must run.
4. ****Evidence or it didn't happen.**** Every finding quotes the kit (Part/section, with the quoted text) or states ABSENT explicitly.
5. ****Licensing gate.**** The split must hold end to end: addon/ and layer/ permissive (MIT per D-03), operator/ and repository default AGPL-3.0-or-later. Verify dependency-compatibility concretely against the §4.2 register, not in the abstract. Verify the FSF combined-works reasoning in §4.1 against primary FSF sources before it appears in any legal notice (the kit itself marks this NOT VERIFIED).
6. ****Vendor gate, three tiers.**** No Meta, OpenAI, or xAI – direct or transitive: SDKs, models, model *lineage and training data*, APIs, infra. Google permitted. Distinguish (a) technical dependency – Blocker; (b) funding/governance proximity – finding requiring explicit human decision (kit D-02 is the worked example); (c) no relationship. Model provenance includes base-model lineage and instruction-data provenance, not just the licence string on the card.
7. ****Rule 9.**** Amendments are proposals until approved. Recommend nothing destructive or irreversible for execution without explicit go-ahead.
8. ****Web research is expected**** wherever the kit is unverifiable, thin, or dated. Primary sources, publication dates, a per-phase source ledger. Distinguish "I checked and found nothing" from "I didn't check."
9. ****Terse.**** No hyperbole, no praise, no filler, no restating the kit back. Findings and evidence only.
10. ****One phase per response.**** End each phase with its ledger and STOP. Wait for G0.
11. ****The kit's self-audit is evidence, not verdict.**** For every F-### you rely on, state whether you re-confirmed it (and how) or are adopting it unverified. Any F-### you refute is a finding against the kit.
12. ****Seeded suspects are questions, not answers.**** Annex A lists locations and suspicions. Confirm, refute, or re-severity each with evidence. A cleared suspect goes in the closest-calls list with the reasoning.

Finding format (use everywhere)

F-### | Severity | Category | Location | Evidence (quote or ABSENT) | Why it matters | Recommended fix | Confidence (H/M/L)

****Severities.**** BLOCKER – build fails, product is wrong, legal/security/licensing breach, or a cheap-to-avoid irreversible decision. MAJOR – significant rework, cost, or risk if not fixed pre-build. MINOR – fix during build. NIT – polish.

IDs are stable for the whole audit and must not collide with the kit's F-001...F-030: start at F-101. Amendments are A-## and map to findings. Decisions requiring the human are D-##, continuing the kit's D-01...D-08 numbering.

Project-specific gates (standing, in addition to the operating rules)

- **Loopback gate:** no service anywhere binds anything but 127.0.0.1. Any counterexample is a Blocker.
- **Explicit-layer gate:** the OpenXR API layer is explicit and environment-scoped only (XR_API_LAYER_PATH / XR_ENABLE_API_LAYERS). Any implicit.d manifest, registry write, installer, or persistent system modification is a Blocker – it is the exact failure mode the kit cites from OpenXR-Toolkit's discontinuation.
- **Browser-isolation gate:** no browser-reachable path to the engine port. Any design where a web page the user visits can reach an authenticated-capable engine endpoint is a Blocker.
- **Approval gate:** no mutating action without a human-reviewed diff in v1 (kit D-06). Any auto-apply path, flag, or bypass is a Blocker.
- **Tier-honesty gate:** every capture and injection carries its fidelity tier through to the report. Any code path where a result can claim a fidelity it did not have is a Blocker.

Phases

Phase 0 – Inventory, provenance & scope contract

Enumerate every KB file: name, one-line purpose, freshness signal, inter-document dependencies. Map the kit's Part structure (0–10) and its provenance conventions. State what the kit claims the project is (thesis, user, outcome) in ≤5 lines – from evidence.

Name the documents you expected but did not find: a demo project specification, hardware requirements, an install/onboarding runbook, a data-retention policy for run artifacts and audit logs, a security disclosure policy, a versioning/release policy beyond the wire protocol.

Verify the document-control block: does the kit's claimed status ("design complete, implementation not started") hold against the KB contents?

Declare the committed stack: Godot 4.7.x, GDScript, Python 3.11+, C++17/CMake, OpenXR 1.1, Ollama, uv, gdUnit4, GitHub Actions.

State what this audit will and won't cover.

Output: inventory table, expected-but-absent list, scope contract. STOP.

Phase 1 – Claims, sources, epistemic audit & open-items closure

Extract every load-bearing factual claim: engine APIs, issue status, protocol behaviour, licences, model provenance, tool versions, browser behaviour.

Re-verify each against primary sources **as of today**, or mark FALSE / STALE / NOT ATTEMPTED with dates. Annex B is the mandatory re-verification schedule – every row, no skips. Flag citations that do not say what the kit says they say.

Close the kit's open items. Work Part 10.2 (BLOCKED) and 10.3 (NOT ATTEMPTED) item by item. For each: close it with a primary source, re-attempt and record the new failure mode, or carry it forward with a stated reason. Highest priority: the OpenXR-MotionCompensation hooked-function list (clone the repo; A2's design rests on it); the FSF AGPL FAQ wording on §13 and plug-ins; licence files and activity of the five candidate Godot MCP servers (gates D-07); the OMBI SOM normative-spec question (gates D-08); exact licence strings on the Ollama model cards for every model in §4.3.

Pay specific attention to: whether the three architectural load-bearers still hold – (a) issue #100004 open and unfixed, (b) EditorInterface unavailable in the running game process, (c) WebSocketPeer exposing no inbound Origin header. If any has changed, STOP the audit and report immediately: the architecture would need revisiting and every downstream phase is invalidated.

Output: claims table, Annex B completed, open-items disposition table, source ledger, findings. STOP.

Phase 2 – Logic, models, assumptions & the decision register

Contradiction hunt across all Parts: definitions, numbers, scope, naming, versioning. Prose-vs-prose and prose-vs-code (Part 6, §7.2) alike.

Surface hidden assumptions; rank the five most load-bearing and stress-test each: what breaks if false, detection method, cost of being wrong. Candidates to confirm or displace: the agent's actual work doesn't need the composited frame (D-04's premise); a local 7B VLM is good enough at scene-tree-plus-screenshot reasoning to be useful; developers will tolerate a human-in-the-loop approval step on every mutation; the debugger channel is available in the configurations the kit needs it; one maintainer can keep the layer current across Vulkan/OpenGL/D3D12.

****Re-validate the decision register.**** D-01...D-08: for each, state whether the default still holds given Phase 1's evidence, what new information changes it, and what forcing function should move it from Open.

Audit the tier model as a model: are T1/T2/T3 and A1/A2/A3 genuinely interchangeable at the Driver boundary, or does any flow semantics leak tier-specific assumptions (settle detection across tier switches, selector resolution against T3-only scenes)?

Every "we will X" in the kit must have a stated mechanism. Missing mechanism = finding.

****Output:**** contradiction list, assumption register, decision-register revalidation, tier-model verdict, findings. STOP.

Phase 3 – Architecture, security, infra & SDLC

****Architecture:**** data flows across the three planes, state, failure modes, single points of failure, degraded behaviour. Verify the tier-fallback semantics of §2.8 are actually implementable at the Driver boundary.

****Security & privacy – kit-specific surface, in addition to STRIDE over §3.1:****

- The debugger channel: availability by build type (editor-launched, debug export, release export – the kit marks this INFERRED), and the authentication posture of Godot's remote-debug endpoint. Who else can speak the EngineDebugger protocol to the editor, and what can they make the editor do (filesystem scan, script assignment, scene save)?

- Token lifecycle: generation, storage permissions (see Annex A, S-02), rotation, what happens on token-file deletion, multi-project collisions under user://.

- Audit-log integrity: §3.7 says "append-only." By what mechanism? No hash chaining, signing, or WORM storage is specified. Tamper-evidence is ABSENT until you find it.

- Approval gate UX as a security control: diff-review fatigue is the known failure mode of human-in-the-loop gates. Does anything bound the approval rate, batch related diffs, or detect rubber-stamping?

- Model supply chain: the threat model ends at "the local model server." The weights themselves – Ollama registry provenance, digest pinning, GGUF tampering – are unexamined. ABSENT unless found.

- Egress enforcement: §3.6 requires enforcement outside the process. Is the mechanism specified enough to build (netns/firewall/container per D-05), and does anything verify at runtime that the enforcement is actually active?

****Infra:**** deploy-target fit, the M0-M7 environment story, the resource envelope (minimum GPU/VRAM/RAM for the engine + capture + a 7B-12B Q4 VLM on one workstation; per-agent-step latency budget; behaviour under GPU contention between the XR runtime and inference). This envelope is ABSENT from the kit; its absence is a finding to severity-assess, and you must produce a defensible first draft of the numbers with sources.

****DevOps/SDLC:**** repo layout, CI gates (§7.1) against the workflow that implements them (§7.2) – verify every mandated gate exists in the YAML and every policy the gates encode is honoured by the YAML itself; test pyramid; release process; observability.

Verify the licensing and vendor gates concretely against §4.2, including the removed-but-listed rows.

****Output:**** architecture verdicts, security findings, resource-envelope draft, SDLC gap list, CI gate conformance matrix (§7.1 gate × present/correct in §7.2), findings. STOP.

Phase 4 – Reference implementation audit (Part 6 as code)

Execute Operating Rule 3 across every listing in §6.1–6.8, the manifest, the launch script, and the CMake file.

For each listing: does it parse/compile against the stated toolchain? Are the symbols it references defined somewhere in the kit (class names, autoload names, file paths)? Do prose claims about the listing match what the listing does (Parts 3 and 6 disagree anywhere)? Are the stated prerequisites sufficient?

Cross-check the §2.6 repository layout against Part 6 coverage: which files in the layout have no reference implementation, and does the build plan (Part 8) assume any of them exist? Work Annex A suspects S-01 through S-16 to disposition.

****Output:**** per-listing execution results table (parsed/compiled/failed + evidence), prose-vs-code drift list, coverage gap list, Annex A dispositions, findings. STOP.

Phase 5 – Product, UX & the first five minutes

Reconstruct the users from evidence: who installs this, on what machine, with what XR hardware. Unsupported persona claims are findings.

Walk the onboarding path end to end as the user: install Godot 4.7.x, install the addon (by what mechanism – the §2.6 `addon/` path vs the `res://addons/` convention in §6.7), install uv/Python, install Ollama, pull a multi-GB model, launch the demo, run the first flow, see the first approval diff. Every dead end, unstated prerequisite, or ambiguous step is a finding.

Audit the web UI against §2.1's principles and M6's acceptance: Origin validation, token gate, vendored assets, CSP, loading/empty/error states, keyboard navigation, WCAG 2.2 AA intent. Anything the UI promises that the architecture can't deliver is a Blocker.

Audit the approval flow as a product surface: what the human sees, what a rejection does, how a rejected diff is recorded, what happens when the human is away (agent blocked? queued? timed out?).

****Output:**** flow-by-flow findings, onboarding gap list, UX debt list. STOP.

Phase 6 – Unknown unknowns & comparables

Run each technique explicitly, labelled by technique:

****Premortem:**** twelve months post-launch, the project is dead. Three most probable obituaries from evidence.

****Red-team personas:**** hostile power user; prompt-injection adversary who controls a texture or node name in the user's scene; exhausted first-time user; future maintainer inheriting the repo; hostile platform (a Godot minor release that breaks an API you depend on – state which API is most exposed); well-funded competitor. One attack paragraph each.

****Comparables scan (research):**** Maestro (living; what its settle/wait design teaches, what its scripting-engine migration warns), OpenXR-Toolkit (dead; the author's stated kill reason and whether the kit's explicit-layer scoping genuinely escapes it), OpenXR-MotionCompensation (living; Windows-only, LGPL-2.1), GFXReconstruct (living; adjacent, not competing), the candidate Godot MCP servers (living/dead; gates D-07). What killed the dead ones; what the living ones all do that this kit ignores.

****Expert-question test:**** the five questions a Godot XR maintainer or an OpenXR working-group member would ask in the first ten minutes that the kit cannot answer.

****Checklist delta:**** standard production-readiness checklist; every absent item.

****Output:**** ranked unknown-unknowns register, each with a mitigation or research item. STOP.

Phase 7 – Build strategy, spec & kickoff prompt

Audit the build strategy: M0–M7 ordering, acceptance criteria, definition of done, dependency ordering, what D-04 defers and what that deferral costs (F-010 is never fully resolved in v1 – is that honestly disclosed to users?).

****Spec completeness test:**** could a competent stranger build this without asking questions? Every question they'd ask is a finding. Include at minimum: the addon install mechanism, the demo project's provenance, the flow runner's CLI surface, and how the web UI is served in production (Flask dev server is not a production answer – find what the kit says, or ABSENT).

Audit the kit's §9.2 kickoff prompt against everything found so far. Then ****rewrite it**** as a standalone block: scope; constraints (licence split, vendor gate, loopback, explicit-layer, approval gate, Rule 9, RPI); repo bootstrap; the corrected CI gate spec from Phase 3; milestone plan with acceptance criteria; test strategy; explicit stop points for human review; and a verification loop that requires executing, not reading, the deliverables.

****Output:**** strategy findings + full rewritten kickoff prompt. STOP.

Phase 8 – Synthesis, self-audit & verdict

Consolidated findings ledger, all phases, deduplicated, cross-referenced to the kit's F-001... F-030 where overlapping (state: re-confirmed / refined / refuted / new).

****Numbered amendments A-01...A-N:**** exact document, exact change, finding(s) resolved. Ordered by dependency, then severity.

****Remediation backlog**** for anything not fixable by amendment, plus decisions for the human as D-09... (continuing the kit's register).

****Audit-the-auditor:**** list every claim you verified by reading rather than execution, every retrieval you skipped, every suspect you cleared on M/L confidence, and every place you adopted a kit finding without re-confirmation. This section is mandatory; its absence invalidates the verdict.

****Verdict:**** READY / READY WITH AMENDMENTS (name the gating ones) / NOT READY (state the kill criteria). Close with the one sentence you'd put at the top of the repo README describing what this is.

Annex A – Seeded suspects (verify or clear; do not trust, do not ignore)

```

| ID | Location | Suspicion to disposition |
|---|---|---|
| S-01 | §7.1 vs §7.2 | The digest-pinning mandate vs the tag-pinned workflow (`@v4`, `@v5`,
`v2.3.8`). Contradiction? |
| S-02 | §3.2 vs §6.1 | "Restrictive permissions" on the token file vs code that sets none;
does GDScript expose any permission API to satisfy the claim? |
| S-03 | §6.1 vs §6.6, Part 6 coverage | `Capture.capture_async` vs `class_name
OperatorCapture` (mismatch?); `inject.gd` has no listing anywhere – the only v1 actuation
tier (A3) lacks a reference implementation while M3 requires it. |
| S-04 | §6.4 | `is_symlink()` after `.resolve()` is unreachable; TOCTOU window between
validation and write. Real, and what severity? |
| S-05 | §2.6 vs §6.7 | `addon/openxr_operator/` vs `res://addons/openxr_operator/` –
install mapping undocumented. |
| S-06 | §6.2 | `num_ctx: 8192` vs a depth-6 scene tree plus base64 image tokens – context
exhaustion in the default configuration? |
| S-07 | §2.5 | Debugger-channel availability in release exports (kit: INFERRED) and the
remote-debug endpoint's auth posture. |
| S-08 | §3.7 | "Append-only" with no integrity mechanism – tamper-evidence ABSENT? |
| S-09 | §7.5 | Model weights absent from supply-chain section: no registry provenance, no
digest pinning for `ollama pull`. |
| S-10 | §7.2 | gitleaks binary downloaded without checksum verification inside the supply-
chain pipeline. |
| S-11 | Whole kit | No minimum-hardware / latency envelope for engine + capture + 7B–12B Q4
VLM on one GPU. |
| S-12 | §4.2 | gdUnit4 "v6.x" – verify the minimum version that actually supports Godot
4.7/4.7.1 against the project's compatibility table. |
| S-13 | §4.3 | The Qwen3-VL caution note cites a March 2026 issue – verify its current
status before the warning is kept or dropped. |
| S-14 | §3.4, Part 5 | Approval-gate fatigue: no rate bounds, no batching, no rubber-stamp
detection. |
| S-15 | §6.5 | Settle detector: verify the timeout is honoured on every path (Maestro
shipped a bug where it wasn't – the kit says so itself) and that `on_timeout` values other
than 'proceed'/'fail' are rejected. |
| S-16 | §6.1 | Constant-time compare returns early on length mismatch; rate-window
accounting under exactly-boundary timing; auth deadline measured from accept, not from
handshake completion. Nits or worse? |

```

Annex B – Claim re-verification schedule (complete every row, dated as of audit day)

```

| # | Kit claim | Kit location | Re-verify against |
|---|---|---|---|
| B-1 | Issue #100004 open, unfixed in 4.7 | §0.2, §2.3 | The issue and its linked PR, today |
|
| B-2 | EditorInterface unavailable in running game | §1.2 F-011, §2.5 | Current class reference |
| B-3 | WebSocketPeer exposes no inbound Origin header | §3.2 | Current class reference + websocket module source |
| B-4 | Godot 4.7.x is the right target (vs 4.7.1, vs 4.8) | Document control | godotengine.org release pages |
| B-5 | Chrome LNA WebSocket enforcement version and browser coverage | §3.2 | Chrome release notes / developer.chrome.com |
| B-6 | Ollama schema-constrained decoding with vision models | §3.5, §6.2 | docs.ollama.com |
| B-7 | Every §4.3 model: licence, lineage, gate verdict | §4.3 | Current model cards, base-model licences |
| B-8 | XR_EXT_conformance_automation runtime support | §2.4 | Khronos registry + Monado releases |
| B-9 | Layer discovery via XR_API_LAYER_PATH / XR_ENABLE_API_LAYERS | §2.3 | Current loader specification |
| B-10 | gdUnit4 / gdtoolkit / uv / osv-scanner / gitleaks versions and compat | §4.2, §7.1 | Project release pages |
| B-11 | Trivy GHSA-69fq-xp46-6x23 and safe versions | §4.2 | GitHub advisory database |
| B-12 | SLSA version, attestation action status, SPDX/CycloneDX versions | §7.5 | slsa.dev, spdx.dev, CycloneDX spec |
| B-13 | WCAG 2.2 remains the operative Recommendation | F-030, M6 | w3.org |
| B-14 | Meta-W4 Games and Khronos-Godot OpenXR funding proximity | §4.5 | Primary announcements |
| B-15 | Maestro design details (settle defaults, GraalJS migration, issue #2843) | Part 5 | docs.maestro.dev + changelog |
| B-16 | OpenXR-Toolkit discontinuation reasoning; MotionCompensation mechanism | §2.3, §2.4 | Author sources; cloned repo |

```

What good looks like

No finding without quoted or explicitly-absent evidence.
 No CONFIRMED claim without a source, a date, and – for code claims – an execution result.
 Every Annex A suspect dispositioned; every Annex B row dated.
 The kit's own findings ledger is re-derived, not inherited.
 The rewritten kickoff prompt is runnable as-is once amendments land.
 If a phase genuinely checks out, say so in one line and show the three closest calls you cleared.

Footnotes

[^1^]: <https://github.com/godotengine/godot/issues/100004> – "Viewport Texture doesn't work with output override (OpenXR)", filed 3 Dec 2024, open as of 9 Aug 2026.

[^2^]: <https://godotengine.org/blog/release/> – Godot 4.7 "Lights, Camera, Action!" stable, 18 Jun 2026.

[^3^]: <https://godotengine.org/article/maintenance-release-godot-4-7-1/> – Godot 4.7.1 maintenance release, 14 Jul 2026.

[^4^]: <https://myconnectionserver.visualware.com/support/v11/userguide/chrome-lna-websocket> – Chrome 147+ LNA WebSocket enforcement (Apr 2026); Firefox/Safari not enforcing. Corroborates developer.chrome.com LNA guidance.

[^5^]: <https://docs.ollama.com/capabilities/structured-outputs> – JSON Schema in `format`, vision-model support, cloud tier unsupported (retrieved 9 Aug 2026).

[^6^]: <https://ollama.com/blog/qwen3-vl> – Qwen3-VL on Ollama, 14 Oct 2025.

[^7^]: <https://github.com/openclaw/openclaw/issues/50569> – Ollama vision-model detection issue; fixes recorded in 2026.4.x releases.

[^8^]: <https://www.collabora.com/news-and-blog/news-and-events/monado-25-1-0-enabling-tomorrows-openxr-experiences.html> – Monado 25.1.0, 10 Dec 2025.

[^9^]: <https://vr.org/articles/monado-open-source-openxr-runtime-powering-xr-industry-2026> – Khronos-funded Monado Upgrade Project including XR_EXT_conformance_automation, 24 Jun 2026.

[^10^]: <https://github.com/godot-gdunit-labs/gdUnit4> – compatibility table: v6.1.x ≤ Godot 4.6.3; Godot 4.7/4.7.1 requires ≥ v6.2.0 (v6.2.1 current).

[^11^]: <https://www.vervali.com/blog/wcag-3-0-accessibility-testing-compliance-2026-standards-timeline-tools-and-how-to-prepare-your-stack/> – WCAG 2.2 operative; WCAG 3.0 Working Draft (Mar 2026), Recommendation ~2029, XR in scope.

[^12^]: <https://www.qualiboost.com/resources/wcag-22-vs-30-whats-coming/> – WCAG 2.2 vs 3.0 status, Jul 2026.

Appendix B — Production implementation prompt v2.0 (full text)

Paste verbatim into a single Kimi Code session at the repository root for a non-swarm build. This is the fallback that degrades the Part 5 topology to one session with plan/explore/coder sub-agents.

```
# Production Implementation & Hardening – OpenXR Operator (v2.0)
```

```
**What this file is.** A hardened, domain-specialized successor to the generic *Production-ready code review and optimization* prompt, adapted to the OpenXR Operator project and wired to its companion audit (*Adversarial QA & Production-Readiness Audit – OpenXR Operator v3.0*). It supersedes both the generic prompt and the build kit's own §9.2 kickoff prompt. Section 1 documents what changed and why. Section 2 is the research baseline, verified 9 August 2026. Section 3 is the paste-ready implementation prompt.
```

```
**Relationship to the audit prompt.** The audit prompt (v3.0) finds everything that would fail; this prompt builds and hardens. They share the epistemic model, the gates, and the finding/amendment ledger format, so the outputs of one are machine-consumable inputs to the other. Run the audit first; this prompt consumes its amendments. If no audit has been run, Phase 0 below does a compressed one before any code is written.
```

```
---
```

```
## 1. Adaptation analysis – what v2.0 changes and why
```

```
### 1.1 Holes in the generic prompt (fixed)
```

```
| Hole | Fix in v2.0 |
|---|---|
| Web-app assumptions baked into the DoD (console errors, `dangerouslySetInnerHTML`, responsive breakpoints, Cloudflare Pages default). Meaningless for a Godot addon + Python agent + C++ OpenXR layer. | DoD rewritten per plane: engine (GDScript parse, debugger errors), agent (mypy strict, pytest), layer (compiler warnings-as-errors, explicit-layer gate), web UI (the only place CSP/responsive/ally apply). Cloudflare default removed – the deploy target is a developer workstation. |
| No toolchain preflight. "Build and run it" assumes the toolchain exists and works. For this project the sandbox mechanism itself (bubblewrap) can fail with EPERM on hosts where AppArmor restricts unprivileged user namespaces – silently losing a security layer if unchecked. | New Phase 0 – Preflight: verify every tool on the actual host, including a bubblewrap smoke test, GPU/VRAM inventory against the resource envelope, and a hardware-floor check before any planning. Sandbox degradation must be loud, never silent. |
| "UNTESTED" is the only epistemic label. | Adopts the four-state model (CONFIRMED / INFERRED / BLOCKED / NOT ATTEMPTED) from the audit, plus UNTESTED for build claims. Presenting NOT ATTEMPTED as BLOCKED is a report defect. |
| No re-verification of time-sensitive stack claims. "Verify current best practices" is one bullet with no checklist. | Phase 1 carries the audit's Annex B re-verification schedule (engine version, issue #100004, LNA enforcement, Ollama structured outputs, model licences, tool versions) – every row dated as of build day. |
| "No approval gate unless Rule 9 triggers" is acceptable for a web prototype but wrong here: this project's entire security model is a human-in-the-loop approval gate for mutating agent actions. | The approval gate is a product feature under construction, not a process inconvenience. It must be built, tested to be unbypassable, and never flagged off (kit D-06). Separate explicit human stop points are added for M7/D-04 and gate changes. |
| Verification loop is web-centric ("play through every user flow", "no console errors"). | Rewritten: every declarative flow in `flows/`, model-disabled (deterministic) then model-enabled; per-step capture tier and settle outcome recorded; engine debugger monitored for errors. The generic prompt's own flaky-async note is made concrete: XR/frame-timing surfaces get three consecutive clean runs; static surfaces two. |
| No supply-chain execution detail. "osv-scanner, never Trivy" is inherited policy with no teeth. | Pin every GitHub Action by commit digest (the audit's S-01), checksum-verify every downloaded binary (S-10), pin model weights by digest (S-09), `uv sync --frozen` everywhere. |
| No relationship to a prior audit or a design kit. The generic prompt assumes a bare repo. | v2.0 consumes three inputs explicitly: the build kit (canonical design), the v3.0 audit outputs (findings F-###, amendments A-##, decisions D-##), and the repo. Amendments apply first, in dependency order. |
```

```
### 1.2 Holes in the kit's own §9.2 kickoff prompt (fixed)
```

1. ****No preflight.**** §9.2 jumps to repo mapping. It never checks that Godot 4.7.x, uv, Ollama, CMake, or the sandbox mechanism actually work on the build host – so a broken bubblewrap or an absent GPU is discovered at M5, not minute 5.
2. ****No resource-envelope step.**** Nothing ever measures VRAM/RAM/latency against a stated floor (audit suspect S-11). Phase 0 and the DoD now require it.
3. ****CI invocation correctness is unexamined.**** §7.2's workflow runs `godot --headless --path demo\_project -s addons/gdUnit4/bin/GdUnitCmdTool.gd` – with `--path demo\_project`, the script path resolves against *that* project, so gdUnit4 must be installed in the demo project, and a fresh CI checkout needs a `--import` pass first or resource import fails. `--check-only` requires `--script` and headless, and surfaces errors only – warnings need `--debug` (which can drop into the debugger and hang CI without a timeout) or gdlint. Phase 0 verifies every CI command actually runs before Phase 3 relies on it.
4. ****No instruction to disposition the audit's seeded suspects.**** §9.2 maps fixes to the kit's F-### ledger only. v2.0 adds the audit's F-1### findings and S-01...S-16 suspects to the plan mapping.
5. ****Model supply chain absent.**** §9.2 never pins model weights. `ollama pull` without a digest is an unpinned dependency in the security-critical path – now a gate.
6. ****Verification loop lacks tier evidence.**** §9.2 logs pass/fail per flow but doesn't require the capture tier and settle outcome per step – the exact data needed to catch a silent fidelity downgrade (the tier-honesty gate).

2. Research baseline addendum (verified 9 August 2026)

Build-mechanics facts this prompt depends on. Re-verify each on build day (Phase 1, Annex B schedule).

#	Fact	Status	Evidence
R-9	`godot --check-only` only parses errors, requires `--script` and (on display-less platforms) `--headless`; warnings require `--debug`, which can drop into the debugger – wrap every Godot CLI call in CI with a timeout.	CONFIRMED	Godot command-line docs; forum confirmation [^1][^2]
R-10	gdUnit4 v6.2.x runs headless via `godot --headless --path <project> -s res://addons/gdUnit4/bin/GdUnitCmdTool.gd` with JUnit/HTML report output; v6.2.0+ is the line that supports Godot 4.7/4.7.1. A fresh checkout needs a `--import` pass before tests.	CONFIRMED	gdUnit4 repo and asset page [^3][^4]
R-11	bubblewrap relies on unprivileged user namespaces; on kernels with `apparmor_restrict_unprivileged_userns=1` (Ubuntu 24.04 line) sandbox preflight can fail EPERM. Preflight test and loud fallback are mandatory; the sandbox policy lives in the bwrap arguments, not the tool.	CONFIRMED	bubblewrap README; reproduced preflight failure [^5][^6]
R-12	`uv sync --frozen` is the CI-standard reproducible install; `uv lock --check` fails on a stale lockfile.	CONFIRMED	uv documentation/workflow references [^7]
R-13	Godot 4.7.1 (14 Jul 2026) is the current maintenance release; pin CI to 4.7.1, state a 4.8 policy.	CONFIRMED	godotengine.org [^8]

3. The implementation prompt (paste-ready)

Everything below the rule is the deliverable prompt. Paste into Claude Code (or equivalent agent) at the repository root.

Mission

Take the OpenXR Operator from design-complete to production-ready, shippable state. Three inputs are canonical, in this order of authority:

1. **The build kit** (`docs/BUILD\_KIT.pdf`, source in `docs/kit/`) – architecture, tier model, security design, licence split. Refine execution, do not redesign.
2. **The v3.0 audit outputs** (`AUDIT.md` findings F-###/F-1##, amendments A-##, decisions D-##) – apply amendments first, in their stated dependency order. If the audit has not been run, run Phase 0's compressed audit before anything else.
3. **The repo as it exists** – actual behaviour outranks both documents where they disagree; log every such disagreement in AUDIT.md.

Where intent is ambiguous, follow the dominant pattern already in the kit and log the ambiguity; do not invent a new direction.

Operating rules

1. **RPI:** Research → Plan → Implement. No code changes until AUDIT.md and PLAN.md exist on disk.
2. **Rule 9:** no destructive or irreversible action (force-push, history rewrite, file/branch deletion, dependency removal) without listing it first and getting explicit approval.
3. **Zero fabrication.** Every claim in every report is backed by something you ran, read, or can cite from a primary source. Labels: CONFIRMED (ran it / primary source, dated), INFERRED (reasoning shown), BLOCKED (what you tried, how it failed), NOT ATTEMPTED (said plainly), UNTESTED (for build claims). Never write "could not confirm" for something you did not attempt.
4. **Licence split is invariant:** `addon/` and `layer/` MIT, `operator/` AGPL-3.0-or-later, repo default AGPL-3.0-or-later. Keep `LICENSES/` and `NOTICE` accurate on every dependency change.
5. **Vendor gate:** no Meta, OpenAI, or xAI dependency, direct or transitive – SDKs, models, model lineage and training-data provenance, APIs, infra. Google permitted. `tools/vendorscan.py` enforces this and must stay green.
6. **Security gates are invariant:** loopback-only binds; explicit environment-scoped OpenXR layer only (never implicit.d, never registry writes); no browser-reachable path to the engine port; no mutating agent action without a human-approved diff. Each is a CI gate; weakening one requires explicit human approval (Rule 9 applies).
7. **Supply chain:** osv-scanner for vulnerabilities – never Trivy (tag-hijack, GHSA-69fq-xp46-6x23). Pin every GitHub Action by commit digest, not tag. Checksum-verify every binary downloaded in CI. Pin the model by digest (`ollama pull model@sha256:...`), record it in the SBOM. `uv sync --frozen` everywhere; CI fails on a stale lockfile.
8. **Executable claims must be executed.** A check verified by reading code is INFERRED. Only a run makes it CONFIRMED.
9. **Terse output.** Findings over narration. No hyperbole.

Phase 0 – Preflight & ingest (read-only, then verify)

```
- **Toolchain preflight.** Verify on this host, recording versions: Godot 4.7.1 (`godot --version`), Python 3.11+, uv, CMake + a C++17 compiler, Ollama (`ollama --version`, `GET /api/tags`), git. Each missing or wrong-version tool is a P0 in AUDIT.md before any planning.
- **Sandbox preflight.** Run the actual sandbox smoke test: `bwrap --unshare-user --ro-bind / / --proc /proc --dev /dev true`. If it fails (EPERM on user namespaces, AppArmor restriction), record it, check `sysctl kernel.unprivileged_usersns_clone` and `/proc/sys/kernel/apparmor_restrict_unprivileged_usersns`, and flag decision D-05 to the human with the evidence. Never let the sandbox degrade silently.
- **Hardware envelope.** Inventory GPU(s), VRAM, RAM. Record against the kit's model table (Qwen2.5-VL-7B ≈ 6–8 GB at Q4; Gemma 3 12B ≈ 8–12 GB). State the measured floor in AUDIT.md; if the host is below it, that is a P0 for the agent milestones, not a surprise at M5.
- **CI command dry-run.** Execute every command the §7.2 workflow will run, locally, before relying on it: `godot --headless --check-only --script` on each addon file (with a timeout wrapper); the gdUnit4 invocation against the demo project including the `--import` pass a fresh checkout needs; `uv lock --check`. Fix the workflow where reality disagrees.
- **Ingest.** Read the kit, the audit outputs, and the repo. List every amendment A-## and its dependency order. List every open decision D-01..D-08: answered, or default adopted – record which, explicitly. D-04 (build the layer in v1?) and D-05 (sandbox mechanism) block milestone planning; get human answers now if not already answered.
- **Compressed audit (only if no v3.0 audit exists).** Run Phases 0–4 of the audit prompt against the kit and repo. Produce AUDIT.md. Do not skip this to "save time"; building an unaudited design is how F-001-class bugs ship.
```

Phase 1 – Research (read-only)

```
- Map the repo: stack per plane (addon / layer / operator / UI), entry points, build/run/test commands, and every user-facing flow (install → launch → first flow → first approval → report).
- Build and run it. Document actual behaviour per flow against the kit's intended behaviour.
- **Re-verify the time-sensitive claims as of today** (the audit's Annex B schedule): current Godot stable and the 4.8 policy; issue #100004 status; EditorInterface and WebSocketPeer API surface; Ollama structured-outputs behaviour on the pinned version; every §4.3 model's licence, lineage, and current Ollama tag; tool versions (uv, osv-scanner, gitleaks, gdUnit4, gdtoolkit); Chrome LNA enforcement state. Anything changed since the kit was dated is a finding, and if any of the three architectural load-bearers changed, STOP and report before writing another line.
- Verify current best practice for each stack component against primary sources; flag anything the kit or repo does that is deprecated or known-vulnerable.
- Write AUDIT.md: findings P0 (security/broken) / P1 (correctness/UX) / P2 (polish), each with file:line and evidence, cross-referenced to kit F-### and audit F-1## where overlapping.
```

Phase 2 – Plan

Write PLAN.md:

```
- Ordered work mapped to: kit milestones M0–M6 (M7 only with explicit approval per D-04), kit findings F-001..F-030, audit findings F-1## and amendments A-##, and the audit's seeded suspects S-01..S-16 (each gets a disposition: fixed, verified-clear, or deferred with reason).
- Acceptance criteria per item, testable by execution.
- Explicit non-goals (the kit's Part 8 non-goals, verbatim, plus anything the audit added). Then proceed – no approval gate except: Rule 9 triggers, any change to a §6 security gate, starting M7, or adopting a model not in the kit's §4.3 pass-list.
```

Phase 3 – Implement

Work PLAN.md in priority order. One logical change per commit; messages reference the finding or amendment ID. No scope creep.

Code standards per plane: GDScript fully typed, gdlint/gdformat clean; Python `mypy --strict`, ruff clean; C++ `-Wall -Wextra -Werror`, no implicit-layer code paths; web UI – no third-party origins, vendored assets, CSP set.

Every document describing code carries the provenance front matter (`sources:`, `verified:`) per kit §7.4.

The model is a client of the flow runner, never a component of it: the runner must pass its whole suite with the model disabled.

Phase 4 – Verification loop

Repeat:

1. Build from clean state (fresh clone or equivalent).
2. Run every gate in the CI gate spec locally: format, lint, parse, types, unit tests, contract tests, vendor gate, licence gate, bind gate, implicit-layer gate, secrets, SBOM, osv-scanner.
3. Run every flow in `flows/` end to end, model-disabled, then model-enabled. Log pass/fail per flow in VERIFICATION.md with evidence – and for every step record the capture tier actually used, the injection tier, and the settle outcome. A run whose steps are all TIMEOUT_PROCEED is not a passing run; it is a run that is not synchronising. Say so.
4. Security spot-checks by execution, every loop: unauthenticated engine connection closes; wrong protocol version closes clearly; a mutating action without approval cannot execute (attempt it); a path-traversal write is rejected and logged; the UI rejects a bad Origin; `grep -rn "0\0\0\0" over `operator/ addon/` is empty.
5. Fix failures.

Exit when every Definition of Done item passes on **two consecutive full runs** – **three** for the flow/smoke and XR-timing surfaces, which are flaky-async by nature. If the same failure survives three fix attempts, stop and write BLOCKERS.md with root-cause analysis instead of thrashing.

Definition of Done

The kit's Part 8 Definition of Done applies verbatim. In addition, and replacing its web-generic items with plane-specific ones:

- Fresh clone builds and runs with the documented commands on the stated minimum hardware; the measured resource envelope (GPU/VRAM/RAM floor, per-step latency) is documented in README and was actually measured, not estimated.
- Zero lint, type, parse, or compiler-warning errors, or documented suppressions with justification. Godot CLI checks ran under a timeout; no CI step can hang in the debugger.
- Every flow in `flows/` completes end to end with no unhandled exception and no engine debugger errors, with per-step tier and settle evidence in the run bundle.
- The approval gate is proven unbypassable by execution: no mutating action path exists that skips it, and the audit log records the attempt when one is tried.
- Audit log is tamper-evident by the mechanism the audit amendment specifies (hash-chained or signed); "append-only" as an unenforced intention does not count.
- Model weights pinned by digest and recorded in the SBOM; all GitHub Actions pinned by commit digest; all CI-downloaded binaries checksum-verified.
- Sandbox preflight is part of the operator CLI startup; if the sandbox is unavailable the agent refuses to run mutating flows and says why.
- Security: osv-scanner clean or accepted risks in `osv-scanner.toml` with reason and expiry; full git history scanned for secrets; token file created with the most restrictive permissions the platform allows, and the claim in the docs matches the code.
- Web UI: loopback bind, Origin allowlist, token gate, vendored assets, CSP with no third-party origins, loading/empty/error states, keyboard-navigable, visible focus, WCAG 2.2 AA baseline pass – verified with the network cable unplugged.
- Docs: README (setup, build, run, tier model, hardware floor, troubleshooting), CHANGELOG (v1.0 → v2.0 → production deltas), SHIP.md. Every document describing code has current provenance front matter.

Human stop points (hard)

Stop and wait for explicit human decision before: starting M7 (D-04); changing any \$6 security gate or its CI enforcement; adopting any model not on the kit's pass-list; any Rule 9 action; shipping with any D-decision still on an unexamined default.

Final report

SHIP.md: what changed; what was tested with evidence (every claim CONFIRMED-by-execution or marked UNTESTED); known limitations; residual risks; install and run steps. Include three tables: the kit's F-001...F-030 with disposition; the audit's F-1##/S-01...S-16 with disposition; the amendments A-## applied, in the order applied. Close with the resource envelope as measured.

Footnotes

[^1]: https://docs.godotengine.org/en/latest/tutorials/editor/command_line_tutorial.html – `--check-only` parses errors only, requires `--script`; `--headless` required on display-less platforms.

[^2]: <https://forum.godotengine.org/t/cli-showing-gdscript-warnings/132160> – warnings require `--debug`; debugger hang workaround via timeout (Jan 2026).

[^3]: <https://github.com/godot-gdunit-labs/gdUnit4> – CLI tool, JUnit/HTML reports, GitHub Action; compatibility table (4.7.x requires ≥ v6.2.0).

[^4]: <https://godotengine.org/asset-library/asset/4390> – gdUnit4 v6.2.0 asset page (retrieved 9 Aug 2026).

[^5]: <https://github.com/containers/bubblewrap> – user-namespace dependency; security policy lives in bwrap arguments; setuid mode removed.

[^6]: <https://forum.cursor.com/t/agent-cli-linux-sandbox-preflight-fails-unshare-eperm-unless-run-under-strace-apparmor-restrict-unprivileged-usersns-1/160039> – reproduced EPERM preflight failure with `apparmor\_restrict\_unprivileged\_usersns=1` (May 2026).

[^7]: <https://meshworld.in/blog/cheatsheets/uv-cheatsheet/> – `uv sync --frozen` as the CI command; `uv lock --check` for stale lockfiles (2026).

[^8]: <https://godotengine.org/article/maintenance-release-godot-4-7-1/> – Godot 4.7.1 maintenance release, 14 Jul 2026.

References

1. Godot Engine issue #100004: "Viewport Texture doesn't work with output override (OpenXR)". Filed 3 Dec 2024; open as of 9 Aug 2026. github.com/godotengine/godot/issues/100004
2. Godot Engine. "Godot 4.7 — Lights, Camera, Action!" stable release, 18 Jun 2026. godotengine.org
3. Godot Engine. Maintenance release Godot 4.7.1, 14 Jul 2026. godotengine.org/article/maintenance-release-godot-4-7-1/
4. Chrome for Developers / LNA guidance. Local Network Access WebSocket enforcement in Chrome 147+ (Apr 2026); Firefox and Safari not enforcing. developer.chrome.com; corroborated via vendor LNA WebSocket documentation, retrieved 9 Aug 2026.
5. Ollama documentation. Structured outputs: JSON Schema in format, vision-model support, cloud tier unsupported. docs.ollama.com/capabilities/structured-outputs, retrieved 9 Aug 2026.
6. Ollama blog. "Qwen3-VL", 14 Oct 2025. ollama.com/blog/qwen3-vl
7. openclaw issue #50569. Ollama vision-model detection; fixes recorded in 2026.4.x releases. github.com/openclaw/openclaw/issues/50569
8. bubblewrap README. User-namespace dependency; policy in bwrap arguments. github.com/containers/bubblewrap
9. Reproduced sandbox preflight EPERM failure with `apparmor_restrict_unprivileged_userns=1` (forum report, May 2026). forum.cursor.com/t/160039
10. gdUnit4 repository and compatibility table: $v6.1.x \leq \text{Godot } 4.6.3$; Godot 4.7/4.7.1 requires $\geq v6.2.0$. github.com/godot-gdunit-labs/gdUnit4; asset page godotengine.org/asset-library/asset/4390
11. Collabora. "Monado 25.1.0 — Enabling Tomorrow's OpenXR Experiences", 10 Dec 2025. collabora.com
12. vr.org. "Monado: Open Source OpenXR Runtime Powering the XR Industry 2026" — Khronos-funded Monado Upgrade Project incl. `XR_EXT_conformance_automation`, 24 Jun 2026.
13. WCAG 3.0 status reporting (2026): WCAG 2.2 remains the operative W3C Recommendation; 3.0 is a Working Draft (Mar 2026), Recommendation ~2029, XR in scope.
14. Godot documentation. Command line tutorial: `--check-only` parses errors only, requires `--script`; `--headless` on display-less platforms. docs.godotengine.org
15. Godot Forum. "CLI showing GDScript warnings" — warnings require `--debug`; timeout workaround for debugger hangs (Jan 2026). forum.godotengine.org/t/132160
16. gdUnit4 documentation. Headless CI invocation and report output; `fresh-checkout --import pass`. github.com/godot-gdunit-labs/gdUnit4
17. Code Agent Swarm guide. "Run Multiple Kimi Agents in Parallel (2026)": CLI sessions are independent agents; sessions on disk under `~/.kimi-code/sessions/`; `/title`, `--continue`; built-in `coder/explore/plan` sub-agents; `swarm` = multiple sessions coordinated on disk. codeagentswarm.com, 18 Jul 2026.
18. MoonshotAI/kimi-cli issues #746 and discussion #969. Feature requests: Agent Swarm support for Kimi Code CLI — CLI is single-agent; swarm only in web UI (beta). github.com/MoonshotAI/kimi-cli, Jan–Feb 2026.
19. Willison, S. "Kimi K2.5: Visual Agentic Intelligence", 27 Jan 2026 — self-directed agent swarm, up to 100 sub-agents and 1,500 tool calls, up to $4.5\times$ faster. simonwillison.net
20. sulat.com. "Inside Claude Code's agent teams and Kimi K2.5's agent swarm", 6 Feb 2026 — PARL reward structure; orchestrator as learned policy.

References

21. MarkTechPost. "Moonshot AI Releases Kimi K2.6...", 20 Apr 2026 — Agent Swarm scaling to 300 sub-agents / 4,000 coordinated steps; K2.6 release details.
22. uv documentation/workflow references: `uv sync --frozen` as the CI command; `uv lock --check for stale lockfiles` (2026). docs.astral.sh/uv